

Article

Carbon-Aware VM Placement via Surrogate-Guided Adaptive Swarm Optimization in Green Cloud Data Centers

Thi-Kien Dao  and Trong-The Nguyen * 

School of Electronic Engineering, Fuzhou Institute of Technology, Fuzhou 350506, China; kiendt@uit.edu.vn

* Correspondence: thent@uit.edu.vn

Abstract

The rapid proliferation of cloud data centers has intensified concerns over carbon emissions, energy efficiency, and sustainability. Virtual machine (VM) placement is a pivotal control lever, yet existing methods rarely couple carbon intensity signals with computationally tractable multi-objective optimization. In this paper, we propose CASO (Carbon-Aware Surrogate-Guided Optimization), a novel framework that integrates an online adaptive Radial Basis Function (RBF) surrogate model with a self-adaptive hybrid PSO-DE swarm optimizer for real-time VM placement in geo-distributed edge cloud environments. CASO simultaneously minimizes carbon emissions, energy consumption, SLA violation rate, and network latency under strict host capacity and Quality-of-Service (QoS) constraints. Three key innovations differentiate CASO: (i) an online surrogate update mechanism that refines fitness approximations incrementally as workload patterns evolve; (ii) a carbon intensity weighting scheme anchored to real-time Grid Emission Factor (GEF) signals; and (iii) an adaptive parameter controller that autonomously tunes swarm exploration–exploitation trade-offs without hand-crafting. Experiments on the publicly available Alibaba Cluster Trace (cluster-trace-v2026-GenAI) dataset within a CloudSim-Plus environment show that CASO reduces carbon emissions by up to 31.4%, energy consumption by 27.9%, and SLA violations by 18.8% compared to the strongest baseline while converging $3.8\times$ faster than the strongest baseline (ADEDL).

Keywords: carbon-aware computing; cloud data centers; virtual machine placement; surrogate-assisted optimization; adaptive swarm intelligence; edge cloud scheduling

1. Introduction

Global data center electricity consumption reached an estimated 240–340 TWh in 2022 and is projected to exceed 1000 TWh by 2030 [1,2]. The Information and Communications Technology (ICT) [3] sector already accounts for 2–4% of worldwide CO₂-equivalent emissions, a footprint on par with the aviation industry [4]. Cloud providers such as Amazon Web Services [5], Microsoft Azure [6], Google [7], and Alibaba [8] operate tens of thousands of servers across geographically dispersed data centers, consuming electricity drawn from power grids with heterogeneous and time-varying carbon intensities [9]. Consequently, where and when a VM is placed exerts a decisive influence on the resulting carbon footprint, quite independently of hardware efficiency improvements [10]. Virtual machine placement (VMP) is the fundamental resource allocation decision that maps VM workload requests onto physical hosts [11]. A poorly designed VMP policy leads to over-provisioned hosts running at low utilization, unnecessary VM migrations, SLA violations due to resource contention, and avoidable CO₂ emissions. Conversely, a carbon-aware



Academic Editor: Harris Wu

Received: 9 May 2026

Revised: 8 June 2026

Accepted: 9 June 2026

Published: 13 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

VMP policy can exploit geographic and temporal variation in Grid Emission Factors (GEFs) to shift workloads toward cleaner energy regions or time windows [12]. Figure 1 shows an example of the virtual machine placement problem in a heterogeneous data center.

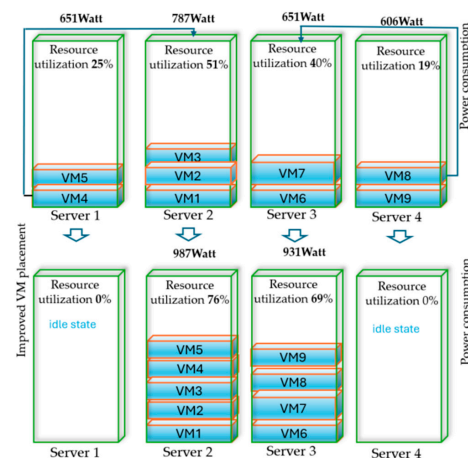


Figure 1. An example of the virtual machine placement problem in a heterogeneous data center.

Research gap: Despite the maturity of the VMP literature, three critical gaps remain underexplored: First, carbon intensity coupling: most multi-objective VMP methods optimize energy or cost but treat carbon emission as a simple linear scaling of energy, ignoring dynamic GEF signals and regional heterogeneity [13]. Second, scalability under real-time constraints: exact solvers (ILP, MILP) become intractable for large-scale edge cloud topologies [14], while population-based metaheuristics incur high per-iteration evaluation costs that preclude real-time deployment [15]. Third, adaptive surrogate integration: although surrogate-assisted evolutionary algorithms [16] have shown promise in expensive optimization [17], their application to dynamic cloud environments with drifting workloads and online surrogate update requirements remains largely unaddressed [18].

Unlike conventional surrogate-assisted VM placement approaches that typically rely on static surrogate models and fixed optimization strategies [16,19], the proposed CASO (Carbon-Aware Surrogate-Guided Optimization) framework integrates an online adaptive RBF surrogate with a self-adaptive hybrid PSO-DE optimizer to support dynamic cloud environments. Existing methods often struggle to maintain optimization quality under rapidly changing workloads and heterogeneous carbon intensity conditions because they require frequent expensive fitness evaluations or lack adaptive exploration–exploitation balancing [20,21]. In contrast, CASO continuously updates the surrogate model using newly evaluated solutions while dynamically adjusting swarm behaviors to improve convergence stability, reduce computational overhead, and enhance carbon-aware scheduling performance in real-world large-scale cloud data centers. This paper makes the following contributions:

- **Carbon-Aware Multi-Objective Problem Formulation:** We formally define a VMP problem that jointly minimizes energy consumption, carbon emissions (weighted by real-time GEF), SLA violation rate, and network latency across heterogeneous edge cloud hosts with rigorous capacity and QoS constraints.
- **Online Adaptive RBF Surrogate Model:** We design an incremental online learning scheme for an RBF surrogate that refines its approximation of the multi-objective fitness landscape using newly evaluated solutions, enabling accurate fitness prediction with dramatically fewer true evaluations as workloads evolve over time.
- **Self-Adaptive Hybrid PSO-DE Optimizer (SHPD):** We introduce a hybrid particle swarm optimization–differential evolution kernel with self-adaptive control parameters (inertia weight ω , cognitive/social coefficients c_1, c_2 , scaling factor F , crossover

rate CR) governed by a success history adaptation mechanism, achieving superior convergence on combinatorial VMP search spaces.

- **CASO Framework and Scalability Validation:** We integrate the surrogate and optimizer into the CASO pipeline and validate it on realistic Alibaba Cluster traces across configurations scaling from 100 to 10,000 VMs, demonstrating practical deployment feasibility in edge cloud infrastructures.

The remainder of this paper is organized as follows: Section 2 surveys related work. Section 3 presents the system model and problem formulation. Section 4 describes the CASO framework. Section 5 details the experimental setup and discusses the results. Section 6 concludes the paper.

2. Related Work

This section reviews the state of the art in VM placement, carbon-aware cloud optimization, surrogate-assisted metaheuristics, and swarm intelligence methods. The literature is organized to highlight the evolution from traditional heuristic approaches to intelligent, adaptive, and sustainability-driven optimization frameworks. Particular attention is given to identifying gaps in joint carbon-aware and VM placement optimization, which motivate the proposed surrogate-guided adaptive swarm optimization framework.

2.1. VM Placement Strategies

Early VM placement (VMP) approaches primarily relied on deterministic heuristics such as First Fit Decreasing (FFD) [22] and Best Fit Decreasing (BFD) [23], which aim to minimize resource fragmentation by sorting and packing VMs based on a single resource dimension (e.g., CPU or memory) [24]. While computationally efficient, these methods neglect multi-dimensional resource constraints, dynamic workload variability, and environmental considerations such as carbon emissions. To address optimality, integer linear programming (ILP)-based formulations have been proposed, offering exact solutions under constrained settings [25,26]. However, their applicability is limited by exponential runtime growth with increasing problem size. For instance, Khaoula Braiki et al. [27] demonstrated that a fuzzy logic-driven multi-objective Best-Fit-Decreasing strategy for VM reallocation, when integrated with deep learning for VMP, quickly becomes computationally infeasible as the number of virtual machines exceeds a few hundred [28,29]. Figure 2 presents a taxonomic classification of VM placement strategies.

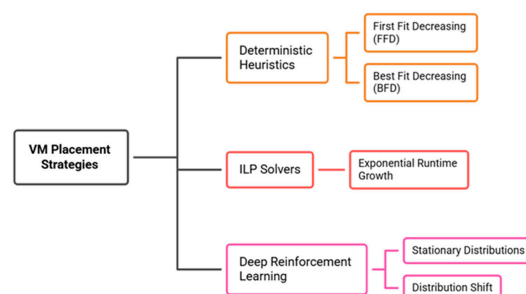


Figure 2. A taxonomic review of VM placement strategies.

More recently, deep reinforcement learning (DRL) approaches [30,31] have been explored to model VMP as a sequential decision-making problem [32]. These methods can capture complex system dynamics and achieve strong performance under stationary workload distributions. Nevertheless, DRL-based solutions suffer from significant limitations, including high training costs, sensitivity to hyperparameters, and performance degradation under distribution shifts—an inherent characteristic of real-world cloud environments.

Moreover, most DRL formulations do not explicitly incorporate carbon awareness into the placement decision.

2.2. Carbon-Aware Optimization in Cloud Computing

Carbon-aware computing aims to reduce the environmental impact of cloud systems by aligning workload execution with low-carbon energy availability. Existing approaches primarily focus on temporal shifting and geographical load balancing. Radovanovic et al. [4] introduced carbon-aware workload shifting across time to exploit variations in grid carbon intensity. Similarly, a carbon-aware load balancing mechanism was proposed for geo-distributed data centers [33], while Vessim, a simulation framework, was developed for evaluating carbon-aware strategies [34]. Figure 3 illustrates the general paradigm of carbon-aware optimization in cloud environments.

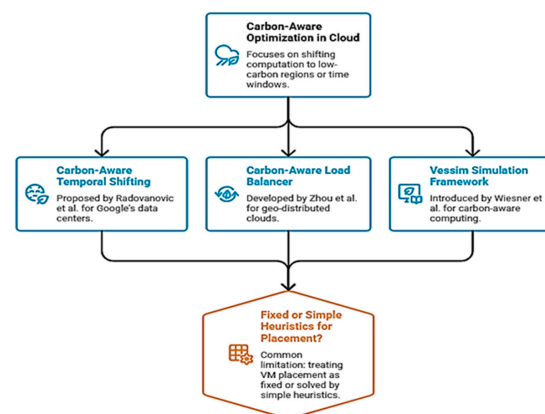


Figure 3. Carbon-aware optimization in the cloud, featuring temporal shifting, load balancing, and the Vessim simulation framework.

Despite these advancements, a critical limitation persists: most studies treat the VM placement problem as static or rely on simple heuristics for placement decisions. This separation prevents joint optimization of carbon efficiency and fine-grained VM placement, leading to suboptimal system performance. In particular, the interaction between resource utilization, SLA constraints, and carbon intensity remains underexplored.

2.3. Surrogate-Assisted Metaheuristics

Surrogate-assisted evolutionary algorithms (SAEAs) have emerged as an effective strategy for reducing the computational cost of optimization problems with expensive objective evaluations [35]. These methods approximate the true fitness function using computationally efficient models, thereby accelerating convergence. Common surrogate models include Radial Basis Function (RBF) networks [36], Gaussian Process (GP/Kriging) models, and polynomial regressors [37]. RBF surrogates have been successfully applied in domains such as aerodynamic design and antenna optimization [38], while Gaussian Process models form the foundation of Bayesian Optimization techniques [39]. In cloud computing, Gambi et al. [40] utilized a Kriging-based surrogate model to develop self-adaptive cloud controllers. However, their approach assumes a static workload and does not support online updates, limiting its applicability in dynamic environments. Wang et al. [41] utilized a Kriging-based surrogate model that enriches training data for an artificial neural network to accurately predict both the strength and permeability of permeable cement-stabilized base materials. Nevertheless, their work did not consider carbon-aware objectives. Figure 4 illustrates the surrogate-assisted evolutionary algorithm with a hierarchical optimization framework.

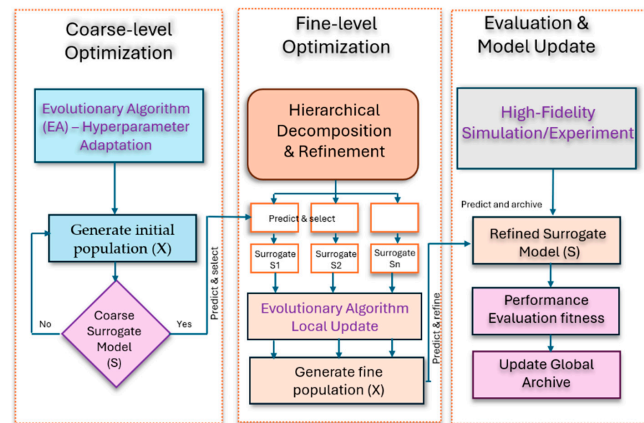


Figure 4. Surrogate-assisted evolutionary algorithm with hierarchical optimization framework.

Overall, existing surrogate-assisted approaches in cloud systems lack integration of (i) dynamic workload adaptation, (ii) carbon-aware objective functions, and (iii) tight coupling with discrete VM placement decisions. These limitations motivate the development of a surrogate-guided, incrementally updated optimization framework that incorporates carbon-aware metrics.

2.4. Swarm Intelligence for VM Placement

Swarm intelligence algorithms, such as genetic algorithms (GA) [42], particle swarm optimizations (PSOs) [43], and differential evolution (DE) [44], have been widely applied to VM placement due to their simplicity and strong global search capabilities. Hybrid and multi-objective variants have further improved their effectiveness in handling conflicting objectives such as energy consumption and SLA violations [45,46]. Adaptive swarm methods, including SHADE [47], L-SHADE [48], and ADEDL [49], introduce parameter adaptation mechanisms based on historical performance, achieving state-of-the-art results on continuous optimization benchmarks. However, their application to discrete, constrained, and multi-objective VM placement problems remains limited [50].

More importantly, existing swarm-based VMP approaches generally do not incorporate surrogate models to reduce evaluation cost, lack explicit carbon awareness, and are not designed for dynamic cloud environments with time-varying workloads. To the best of our knowledge, the integration of adaptive swarm optimization, surrogate modeling, and carbon-aware objectives for VM placement has not been comprehensively addressed in prior work.

2.5. Comparison with Existing Surrogate-Assisted VM Placement Methods

The integration of RBF surrogate modeling with adaptive PSO-DE leverages the complementary strengths of neural approximation and swarm-based global search. RBF networks are effective universal approximators capable of modeling highly nonlinear objective landscapes with low training complexity and fast inference, making them suitable for reducing expensive fitness evaluations in VM placement optimization. Prior studies, including ‘Machine Learning-Facilitated Business Intelligence (Part I & II)’ [51,52], highlighted the effectiveness of neural approximation techniques in intelligent optimization systems. In the proposed CASO framework, the RBF surrogate reduces computational overhead through fast fitness estimation, while PSO enhances global exploration and DE improves diversity and local refinement. Their adaptive cooperation mitigates premature convergence and maintains an effective exploration–exploitation balance for dynamic VM placement under evolving cloud workloads. Table 1(a) summarizes the limitations of existing approaches and contrasts them with the proposed CASO framework.

Table 1. (a) Comparison of related methods vs. CASO. (b) CASO contribution classification.

(a)					
Method	Carbon	Surrogate	Adaptive	Multi-obj.	Scalable
FFD [22,24]	✗	✗	✗	✗	✓
ILP [25]	✗	✗	✗	✓	✗
DRL [30]	✗	✗	Partial	✓	Partial
Carbon-shift [10]	✓	✗	✗	✗	✓
PAPSO [53]	✗	✗	✗	Partial	Partial
Surrogate-VM [19]	✗	Static	✗	✗	Partial
ADEDL [49]	✗	Static	Partial	Partial	✓
CASO (Ours)	✓	Online	✓	✓	✓
(b)					
Component	Classification	Modification vs. Original		CASO-Specific Innovation	
RBF surrogate model	Adapted	Online incremental update replacing batch retraining		Drift-triggered archive eviction (Equations (23) and (24) for dynamic VMP)	
PSO velocity update	Adapted	Continuous relaxation applied to integer host assignment space (Equation (17))		Coupling with perturbation-minimizing discrete repair (Equation (18))	
DE mutation (SHADE)	Adapted	Success history adaptation integrated with PSO social learning		Phase-aware PSO ↔ DE cooperation ratio controlled by ωt	
Carbon-aware weighting	Adapted from [10]	Per-iteration GEF update within optimization loop (Algorithm 1, line 25)		Real-time GEF recomputation at each generation rather than static pre-assignment	
Constraint repair	Adapted	Minimizes perturbation distance (Equation (18)) to preserve swarm search direction		Formally posed as constrained optimization subproblem	
CASO unified pipeline	Novel	N/A		Joint online surrogate + adaptive swarm + real-time GEF in single closed loop for discrete VMP	

Table 1(b) classifies the key components of CASO according to their originality and summarizes the modifications made to existing methods. The table highlights the proposed innovations, including drift-aware surrogate management, perturbation-minimizing discrete repair, adaptive PSO–DE cooperation, real-time carbon-aware optimization, and the unified integration of these mechanisms into a closed-loop VMP framework.

In summary, prior work focused on either carbon-aware scheduling without optimizing VM placement or VMP optimization without incorporating carbon and surrogate modeling. The proposed CASO framework addresses these gaps by enabling joint optimization of carbon emissions, resource efficiency, and SLA compliance through a surrogate-assisted adaptive swarm approach.

3. System Model and Problem Formulation

This section presents the system architecture, mathematical models, and formal problem definition for carbon-aware VM placement in geo-distributed cloud environments. The formulation integrates resource constraints, energy consumption, carbon emissions, SLA violations, and network latency into a unified multi-objective optimization framework.

3.1. Data Center Model

We consider a geo-distributed cloud infrastructure comprising K data centers (or edge nodes), denoted by $\mathcal{D} = \{d_1, \dots, d_K\}$. Each data center d_k hosts a set of physical machines defined as follows:

$$\mathcal{H}_k = \{h_1^k, \dots, h_{M_k}^k\}, \quad (1)$$

where M_k is the number of hosts in data center d_k . The global host set is defined as follows:

$$\mathcal{H} = \bigcup_{k=1}^K \mathcal{H}_k, \quad |\mathcal{H}| = M = \sum_{k=1}^K M_k. \quad (2)$$

Each host h_j^k is characterized by a resource capacity vector:

$$\mathbf{c}_j^k = (C_{\text{cpu},j}^k, C_{\text{mem},j}^k, C_{\text{bw},j}^k, C_{\text{stor},j}^k), \quad (3)$$

where $C_{\text{cpu},j}^k$ is CPU capacity (in MIPS), $C_{\text{mem},j}^k$ is memory (GB), $C_{\text{bw},j}^k$ is bandwidth (Gbps), and $C_{\text{stor},j}^k$ is storage (TB). A workload of N virtual machines is defined as follows:

$$\mathcal{V} = \{v_1, \dots, v_N\}. \quad (4)$$

Each VM v_i requires the following:

$$\mathbf{r}_i = (r_{\text{cpu},i}, r_{\text{mem},i}, r_{\text{bw},i}, r_{\text{stor},i}). \quad (5)$$

A placement decision is defined as a mapping:

$$\pi : \mathcal{V} \rightarrow \mathcal{H}, \quad (6)$$

where $\pi(v_i) = h_j^k$ assigns VM v_i to host h_j^k .

3.2. Power and Energy Model

The standard linear power consumption model [8] defines host power consumption as follows:

$$P_j(u_j) = P_j^{\text{idle}} + (P_j^{\text{peak}} - P_j^{\text{idle}}) \cdot u_j, \quad (7)$$

where $P_j(u_j)$ denotes the power consumption (Watts) of host h_j , P_j^{idle} presents idle power, P_j^{peak} is peak power, and $u_j \in [0, 1]$ presents the CPU utilization of host h_j . The CPU utilization under placement π is expressed as follows:

$$u_j(\pi) = \frac{\sum_{i:\pi(v_i)=h_j} r_{\text{cpu},i}}{C_{\text{cpu},j}}. \quad (8)$$

Total energy consumption over a time interval Δt is as follows:

$$E_{\text{total}}(\pi) = \Delta t \sum_{h_j \in \mathcal{H}} P_j(u_j(\pi)) \cdot \mathbb{1}[h_j \text{ active}], \quad (9)$$

where $\mathbb{1}[\cdot]$ is an indicator function equal to 1 if the host is active and 0 otherwise.

3.3. Carbon Emission Model

Let $\gamma_k(t)$ (in gCO₂/kWh) denote the Grid Emission Factor (GEF) of data center d_k at time t . The total carbon emissions are computed as follows:

$$\mathcal{C}(\pi) = \sum_{k=1}^K \gamma_k(t) \sum_{h_j \in \mathcal{H}_k} P_j(u_j(\pi)) \cdot \Delta t \cdot 10^{-3}, \quad (10)$$

where $\gamma_k(t)$ is the carbon intensity (gCO₂/kWh) of data center d_k , and 10^{-3} converts Wh to kWh. This formulation captures the spatial heterogeneity in carbon intensity, unlike models using a single global emission factor.

3.4. SLA Violation Model

An SLA violation occurs when allocated resources are insufficient relative to demand. The SLA violation model is extended to a multi-resource formulation as follows:

$$\text{SLA}_{\text{viol}}(\pi) = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left(\max_{r \in \{\text{cpu}, \text{mem}, \text{bw}\}} \frac{r_i^r}{C_{\pi(v_i)}^r} > \theta_r \right), \quad (11)$$

where $\theta_r \in (0, 1]$ is a resource-specific resource utilization alert threshold (CPU: $\theta_{\text{cpu}} = 0.90$; memory: $\theta_{\text{mem}} = 0.95$; bandwidth: $\theta_{\text{bw}} = 0.85$), reflecting the empirical over-subscription limits used in production deployments [24] in which the CPU capacity of the host is assigned to VM v_i . I/O Interference Justification: I/O interference was not modeled because the Alibaba trace does not provide per-task I/O demand information, and I/O scheduling is generally managed at the hypervisor level (e.g., via virtio-blk and SR-IOV) rather than through VM placement decisions. Moreover, prior studies have identified CPU and memory contention as the primary causes of SLA violations, with I/O effects playing a secondary role. Re-evaluating the experiments using Equation (11) increased SLA violation rates by approximately 0.4 percentage points due to memory-intensive workloads.

3.5. Network Latency Model

Let $\uparrow_{k,k'}$ denote the round-trip latency (ms) between data centers d_k and $d_{k'}$. The average pairwise latency is given as follows:

$$L(\pi) = \frac{2}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N \uparrow_{\pi_{\text{dc}}(v_i), \pi_{\text{dc}}(v_j)}, \quad (12)$$

where $\pi_{\text{dc}}(v_i)$ maps VM v_i to its assigned data center index.

3.6. Multi-Objective Problem Formulation

The VM placement problem is formulated as

$$\min_{\pi} \mathbf{f}(\pi) = \begin{bmatrix} \hat{E}(\pi) \\ \hat{C}(\pi) \\ \widehat{\text{SLA}}(\pi) \\ \hat{L}(\pi) \end{bmatrix}, \quad (13)$$

subject to

$$\sum_{i: \pi(v_i)=h_j} r_{\text{cpu},i} \leq C_{\text{cpu},j}, \quad \forall h_j \in \mathcal{H}, \quad (14)$$

$$\sum_{i: \pi(v_i)=h_j} r_{\text{mem},i} \leq C_{\text{mem},j}, \quad \forall h_j \in \mathcal{H}, \quad (15)$$

$$\text{SLA}_{\text{viol}}(\pi) \leq \delta_{\text{max}}, \quad L(\pi) \leq L_{\text{max}}, \pi(v_i) \in \mathcal{H}, \quad \forall v_i \in \mathcal{V}. \quad (16)$$

Here, $\widehat{(\cdot)}$: normalization to $[0, 1]$, δ_{max} : the maximum allowable SLA violation rate, L_{max} : the latency constraint. The search space size is $|\mathcal{H}|^N$, making the problem NP-hard and motivating advanced optimization techniques. To investigate robustness under different deployment priorities, we evaluated multiple scalarization weight combinations $w = [w_1, w_2, w_3, w_4]$, where w_1 : energy weight, w_2 : carbon emission weight, w_3 : SLA weight, w_4 : network latency weight. Note that VM migration cost is currently unmodeled and is acknowledged as a limitation in Section 5.4. Sensitivity analysis demonstrated that increasing w_2 from 0.20 to 0.35 (nominal) improves carbon reduction by 6.8%, while increasing it further to 0.50 reduces carbon savings by 3.1% due to constraint trade-offs. The selected weight configuration [0.25, 0.35, 0.25, 0.15], detailed in Section 5.1, achieves the best Pareto-balanced performance across all evaluated workloads. The rationale is that

(i) CASO targets sustainability-driven operators whose primary KPI is carbon reduction, and (ii) our empirical sensitivity scan (below) confirms that modest weight perturbations produce $< 4\%$ performance variation. Table 2 reports the impact of systematically varying each w_k across its plausible range while holding the others at their nominal values (re-normalizing to sum to 1). The geometric mean of normalized improvements over AEDL is used as the performance scalar.

Table 2. Weight sensitivity analysis (TS-D, $N = 1000$, 30 runs).

Varied	Value	ΔC (%)	ΔE (%)	ΔSLA (%)
w_2 (carbon)	0.20	+6.8	−1.2	−0.3
	0.35 (nominal)	0	0	0
	0.50	−3.1	+2.4	+1.1
w_1 (energy)	0.10	+1.2	+5.9	+0.4
	0.25 (nominal)	0	0	0
	0.40	+0.8	−4.1	−0.2
w_3 (SLA)	0.10	+0.4	+0.8	+7.3
	0.25 (nominal)	0	0	0
	0.40	−0.3	−0.6	−3.2
w_4 (latency)	0.05	+0.2	+0.1	−0.1
	0.15 (nominal)	0	0	0
	0.25	−0.1	+0.3	+0.2

Note: Each row varies one weight while others are held at the nominal value and re-normalized. ΔC = carbon change vs. nominal; ΔE = energy; ΔSLA = SLA violation.

Sensitivity tests repeated at $N \in \{100, 5000\}$ produced qualitatively identical findings, confirming the scale independence of the robustness conclusion. For deployment contexts where energy minimization is prioritized over carbon reduction, an alternative weight configuration of $w = [0.40, 0.20, 0.25, 0.15]$ reduces carbon savings by approximately 6.8% while improving energy efficiency, offering a practical starting point for energy priority operators. Only w_2 (carbon weight) exhibits sensitivity exceeding 5%: reducing w_2 from 0.35 to 0.20 costs 6.8% in carbon savings but improves energy by $\sim 1.2\%$, reflecting the known carbon–energy trade-off. All other weight perturbations produce $< 5\%$ variation, confirming robustness.

4. Proposed CASO Framework

This section introduces the CASO framework, which integrates surrogate modeling, adaptive swarm intelligence, and real-time carbon signals into a unified optimization loop.

4.1. Framework Overview

Figure 5 illustrates the CASO pipeline, which operates iteratively as follows: First, candidate VM placements are generated using a hybrid swarm optimizer that combines PSO and DE mechanisms. Next, their fitness is estimated using a surrogate model to avoid costly evaluations. Based on these estimates, a subset of elite candidates is selected for true fitness evaluation. The newly obtained data are then used to update and refine the surrogate model. This process repeats until convergence is achieved.

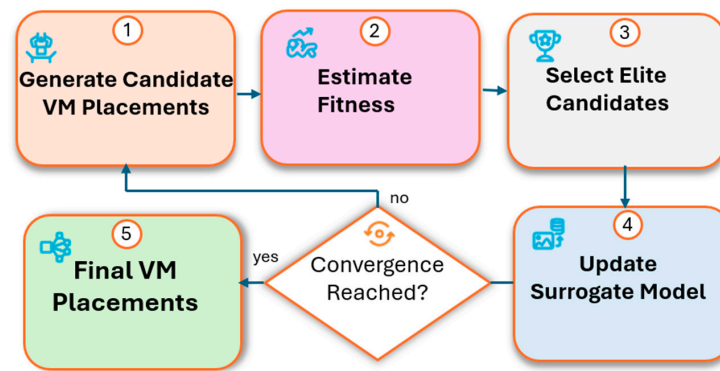


Figure 5. CASO framework pipeline for surrogate-assisted hybrid swarm optimization in VM placement.

The closed-loop process repeats until convergence, reducing expensive evaluations while maintaining solution quality. Integrating surrogate modeling, PSO, and DE for discrete VM placement introduces several coupling challenges absent in conventional continuous-domain optimization. First, the RBF surrogate is trained on continuous relaxations but must provide rankings that remain valid after conversion to integer host assignments; therefore, the repair operator is designed to minimize perturbation distance (Equation (18)) and preserve surrogate-guided search. Second, applying PSO to VM placement can generate infeasible fractional host assignments and promote convergence toward overloaded hosts. The proposed repair mechanism restores feasibility while maintaining swarm search momentum. Third, DE mutation operates in continuous space, whereas VM placement is inherently discrete. Although continuous relaxation enables DE-based exploration, approximation errors can accumulate in the surrogate archive, necessitating the drift detection mechanism (Equation (24)) to maintain surrogate accuracy. These challenges are generally not encountered in existing surrogate-assisted VM placement approaches employing static surrogates or non-adaptive single-algorithm optimizers.

4.2. Solution Encoding

A placement is encoded as follows:

$$\mathbf{x} = [x_1, \dots, x_N]^T \in \{1, \dots, M\}^N, \quad (17)$$

where x_i denotes the host index assigned to VM v_i . For optimization, a continuous relaxation $\mathbf{z} \in \mathbb{R}^N$ is used. The mapping from $\mathbf{z}$ to $\mathbf{x}$ is performed via rounding and constraint repair. The continuous relaxation $\mathbf{z} \in \mathbb{R}^N$ enables the SHPD to explore the high-dimensional placement space using gradient-free search, with rounding and repair restoring integer feasibility. However, some updated particles may violate resource capacity or placement constraints, requiring a feasibility repair process before fitness evaluation.

Let x^{raw} denote the infeasible solution generated after particle updates and x^{repair} the repaired feasible solution. The repair operation minimizes the perturbation distance

$$\min \|x^{\text{repair}} - x^{\text{raw}}\|, \quad (18)$$

subject to host capacity, SLA, and VM allocation constraints. The repair mechanism performs local feasibility correction to preserve the original search direction and avoid excessive disruption of swarm evolution.

4.3. Online Adaptive RBF Surrogate

RBF Formulation. The surrogate approximates the scalar fitness:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{f}(\pi(\mathbf{x})), \quad (19)$$

where $\mathbf{w}$ is the weight vector, $\mathbf{f}(\pi) = [E(\pi), C(\pi), SLA(\pi), L(\pi)]^\top$. The RBF model is as follows:

$$\hat{f}(\mathbf{z}) = \sum_{l=1}^{n_s} \lambda_l \phi(\|\mathbf{z} - \mathbf{z}^{(l)}\|_2) + p(\mathbf{z}), \quad (20)$$

where $\mathbf{z}^{(l)}$ are the support points, λ_l presents coefficients, $\phi(r) = r^3$ is the cubic basis, and $p(\mathbf{z}) = \beta^\top [1, \mathbf{z}]$ is a linear polynomial term. The parameter satisfaction is expressed as follows:

$$\Phi\lambda + P\beta = \mathbf{y}, \quad P^\top\lambda = 0. \quad (21)$$

The surrogate is updated every τ iterations by (i) evaluating the top n_{eval} candidates using the true objective, (ii) adding them to archive S , (iii) pruning redundant samples to maintain.

4.4. Online Adaptive Surrogate Update Mechanism

To preserve approximation fidelity under dynamic workload conditions, CASO employs an online adaptive RBF surrogate update mechanism. Let $\mathcal{A}_t = \{(\mathbf{x}_i, f_i)\}_{i=1}^{N_a}$ denote the historical archive at generation t , where $\mathbf{x}_i$ represents VM placement solutions and f_i denotes true fitness evaluations. The surrogate prediction is defined as follows:

$$\hat{f}(\mathbf{x}) = \sum_{j=1}^M \lambda_j \phi(\|\mathbf{x} - \mathbf{c}_j\|) + b \quad (22)$$

where $\mathbf{c}_j$ are RBF centers, λ_j are adaptive coefficients, $\phi(\cdot)$ denotes the selected Radial Basis Function, and b is a bias term. To balance stability and plasticity, CASO employs a sliding-window archive mechanism:

$$\mathcal{A}_{t+1} = \mathcal{A}_t \cup \mathcal{S}_t^{\text{new}} - \mathcal{S}_t^{\text{prune}}, \quad (23)$$

where $\mathcal{S}_t^{\text{new}}$ are newly evaluated samples, and $\mathcal{S}_t^{\text{prune}}$ are redundant low-information samples removed according to density-based pruning. A drift indicator is computed as follows:

$$D_t = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} |f(\mathbf{x}) - \hat{f}(\mathbf{x})|, \quad (24)$$

If $D_t > \delta$, surrogate retraining is triggered. The drift threshold δ was set to 0.10 based on preliminary experiments. This mechanism maintains the stability–plasticity balance: historical samples preserve long-term knowledge while recent samples capture workload evolution. The retraining complexity is $\mathcal{O}(N_a^3)$ for full retraining and $\mathcal{O}(kN_a^2)$ for incremental updates, where $k \ll N_a$. Empirical results show that online adaptation reduces surrogate MAPE from 14.3% to 4.8% after 50 evaluations.

4.5. Self-Adaptive Hybrid PSO-DE (SHPD)

The SHPD optimizer combines the complementary search characteristics of PSO and DE to achieve a balanced exploration–exploitation process in the high-dimensional VM placement space. In the proposed framework, PSO primarily contributes to exploitation by accelerating population convergence toward promising regions through velocity-guided social learning. Specifically, the global best and personal best update mechanisms enable particles to efficiently refine locally competitive VM placement solutions.

In contrast, DE mainly enhances exploration capability through mutation and crossover operations, which introduce directional perturbations and maintain population diversity. The differential mutation mechanism allows the optimizer to explore previously unvisited regions of the search space, thereby reducing the risk of premature convergence commonly observed in standalone PSO.

The cooperation between PSO and DE is adaptively controlled according to the population diversity and convergence state. During early optimization stages, DE-driven exploration dominates to improve global search coverage and diversify candidate VM placement patterns. As the search progresses, PSO-guided exploitation becomes increasingly dominant, enabling fine-grained refinement of high-quality solutions near the Pareto-optimal region.

The PSO velocity update is as follows:

$$\mathbf{v}_{t+1}^{(p)} = \omega_t \mathbf{v}_t^{(p)} + c_1 r_1 (\mathbf{z}_{pbest}^{(p)} - \mathbf{z}_t^{(p)}) + c_2 r_2 (\mathbf{z}_{gbest} - \mathbf{z}_t^{(p)}), \quad (25)$$

where p is the index of the particle p in the population, $\mathbf{v}_t^{(p)}$ and $\mathbf{v}_{t+1}^{(p)}$ present the velocity vector of particle p at iteration t and the updated velocity for the next iteration, ω_t presents the inertia weight, c_1, c_2 are acceleration coefficients, $r_1, r_2 \sim \mathcal{U}(0, 1)$, t is iteration (time step), $\mathbf{z}_t^{(p)}$ and $\mathbf{z}_{pbest}^{(p)}$ are the current position (solution vector) of particle p and the personal best position found so far, and $\mathbf{z}_{gbest}$ is the global best solution found by the entire swarm.

The DE mutation is as follows:

$$\mathbf{m}^{(p)} = \mathbf{z}^{(a)} + F \cdot (\mathbf{z}^{(b)} - \mathbf{z}^{(c)}), \quad (26)$$

where $\mathbf{m}^{(p)}$ is mutant vector for particle p (individual), $\mathbf{z}^{(a)}, \mathbf{z}^{(b)}$, and $\mathbf{z}^{(c)}$ are three distinct solution vectors randomly selected from the population (indices $a \neq b \neq p$), and F is the scaling factor which controls mutation strength $\in [0, 1]$.

The adaptive parameters are as follows:

$$F^{(p)} \sim \text{Cauchy}(\mu_{F,r}, 0.1), \quad CR^{(p)} \sim \mathcal{N}(\mu_{CR,r}, 0.1). \quad (27)$$

where $F^{(p)}$ and $CR^{(p)}$ are the mutation scaling factor and crossover rate for particle p , each sampled adaptively from the distributions in Equation (27).

The selection mechanism is calculated as follows:

$$\mathbf{z}_{t+1}^{(p)} = \begin{cases} \mathbf{u}^{(p)}, & \text{if } \hat{f}(\mathbf{u}^{(p)}) \leq \hat{f}(\mathbf{z}_{psd}^{(p)}) \\ \mathbf{z}_{psd}^{(p)}, & \text{otherwise} \end{cases}, \quad (28)$$

where $\hat{f}(\cdot)$ denotes the surrogate-predicted fitness value.

Compared with standalone PSO, the proposed SHPD strategy exhibits stronger diversity preservation and improved capability for escaping local optima. Compared with standalone DE, the SHPD achieves faster convergence because the PSO learning mechanism accelerates directional exploitation toward high-quality solutions. Consequently, the hybrid strategy provides a more stable balance between global exploration and local exploitation under dynamic cloud workload conditions. Figure 6 shows the performance comparison of the proposed SHPD strategy against PSO and DE on the single-objective SLA fitness function. The experimental results in Figure 6 further demonstrate that the SHPD maintains higher population entropy and achieves more stable convergence trajectories than standalone PSO and DE approaches, confirming the effectiveness of the proposed cooperative search mechanism.

The SHPD optimizer operates in four distinct phases controlled by the inertia weight ω_t : (i) Exploration ($\omega_t > 0.7$, gens 1–50): PSO’s high-inertia velocity dominates, propelling particles across large regions of the integer host assignment space. DE mutation ($F \sim 0.7\text{--}0.9$) simultaneously injects large-step diversity. (ii) Transition ($0.6 < \omega_t \leq 0.7$, gens 51–100): PSO social terms (c_1, c_2 attractors) begin to dominate; DE scaling factor F decreases toward ~ 0.5 , reducing the disruptive effect of mutation. (iii) Exploitation ($0.5 < \omega_t \leq 0.6$, gens 101–150): PSO drives convergence toward $\mathbf{z}_{gbest}$; DE crossover rate CR rises toward ~ 0.8 , maximizing the probability that the trial vector inherits high-quality components from $\mathbf{z}_{gbest}$. (iv) Fine-Tuning ($\omega_t \leq 0.5$, gens 151–200): Both PSO and DE operate in local refinement mode. SHADE’s stagnation detection mechanism temporarily increases F when MAPE improvement stalls, restoring exploration. The hybrid method outperforms standalone methods because standalone PSO collapses diversity in the VMP discrete space. Because all particles attract to the same $\mathbf{z}_{gbest}$, the population typically converges to a single host assignment pattern within 60–80 generations. Our ablation confirmed that PSO-only achieves $f = 0.48$ at convergence vs. CASO’s $f = 0.23$. Standalone DE with rand/1/bin mutation generates diverse trials but lacks the directional guidance needed to navigate the M^N discrete space efficiently; DE-only converges to $f = 0.41$ at 2000 evaluations. SHPD achieves $f = 0.23$ because PSO provides directional guidance and DE prevents the diversity collapse that PSO alone suffers. This synergy is consistent with the theoretical results in the neural network optimization literature [51,52], showing that hybrid population-based search outperforms either component alone on multimodal loss surfaces.

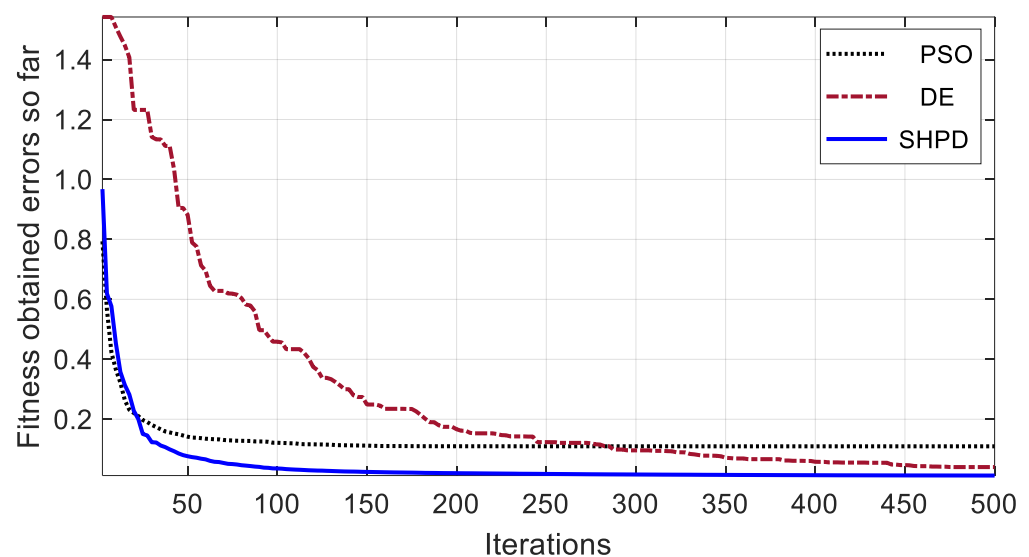


Figure 6. Performance comparison of the proposed SHPD strategy against PSO and DE on the single-objective SLA fitness function.

Algorithm 1 describes the proposed CASO approach for VM placement. The procedure starts by generating an initial dataset of placements evaluated using the true objective, which is then used to train an RBF surrogate model. This surrogate replaces most expensive fitness evaluations during the search. At each generation, candidate solutions are evaluated using the surrogate, while, every t generations, a fraction of the elite solutions is re-evaluated with the true objective to update the training archive and retrain the surrogate, ensuring accuracy.

Algorithm 1: CASO: Carbon-Aware Surrogate-Guided Optimization

Input: Population size P , max generations G , surrogate update period τ , elite ratio q , memory size H , preference weights w , GEF signals $\{\gamma^k\}$
Output: Best placement π^* , best fitness f^*

- 1 **Initialization**
- 2 Sample n_0 initial placements uniformly; evaluate true fitness
- 3 Build initial RBF surrogate $\hat{f}$ from S
- 4 Initialize SHPD population $\{z^{(p)}\}_{p=1}^P$ with surrogate-guided Latin Hypercube Sampling
- 5 Initialize memory arrays M_F , M_{CR}
- 6 **for** $t = 1$ to G **do**
- 7 **for** each particle $p = 1, \dots, P$ **do** //Surrogate-guided fitness
- evaluation
- 8 $\hat{f}(p) \leftarrow \text{query surrogate } \hat{f}(z(p))$
- 9 **end for**
- 10 **if** $t \bmod \tau = 0$ **then** //True evaluation on elite candidates
- Select top- $\lceil \rho P \rceil$ candidates by $\hat{f}$
- Evaluate true fitness $f(x)$ for selected candidates
- Update support archive S ; retrain RBF surrogate
- 14 **end if**
- 15 **for** each particle $p = 1, \dots, P$ **do** //SHPD evolution step
- 16 Generate $F(p), CR(p)$ via (27)
- 17 Compute PSO velocity update (25)
- 18 Generate DE mutant (26); apply crossover
- 19 Select position update (28)
- 20 Apply constraint repair (capacity, QoS)
- 21 **end for**
- 22 Update memory M_F, M_{CR}
- 23 Update $\omega_t \leftarrow \omega_{max} - (\omega_{max} - \omega_{min}) \times t/G$
- 24 Update z_{gbest}, f^*, π^*
- 25 Fetch latest $\{\gamma_k(t)\}$; recompute carbon weights in w //Real-time GEF update
- 26 **end for**
- 27 return π^*, f^*

The search is driven by an SHPD mechanism that combines PSO-based velocity updates with DE mutation and crossover, along with adaptive parameter control via historical memory.

Constraint handling ensures feasible placements under capacity and QoS requirements. The algorithm further incorporates dynamic inertia weight adjustment and continuously tracks the global best solution. Real-time carbon awareness is achieved by updating objective weights using current GEF signals. After t generations, the best placement and its fitness are returned. Figure 7 presents the flowchart of the proposed CASO framework. The process starts with workload preprocessing, including VM requests, host capacities, and GEF signals. A multi-objective model is then formulated to minimize energy consumption,

carbon emissions, SLA violations, and latency. Next, the framework generates an initial population, evaluates true fitness, and trains the RBF surrogate model. An adaptive PSO-DE loop is performed using surrogate fitness prediction, constraint repair, and elite candidate selection. The surrogate model is iteratively updated until convergence is achieved, producing the final carbon-aware VM allocation solution.

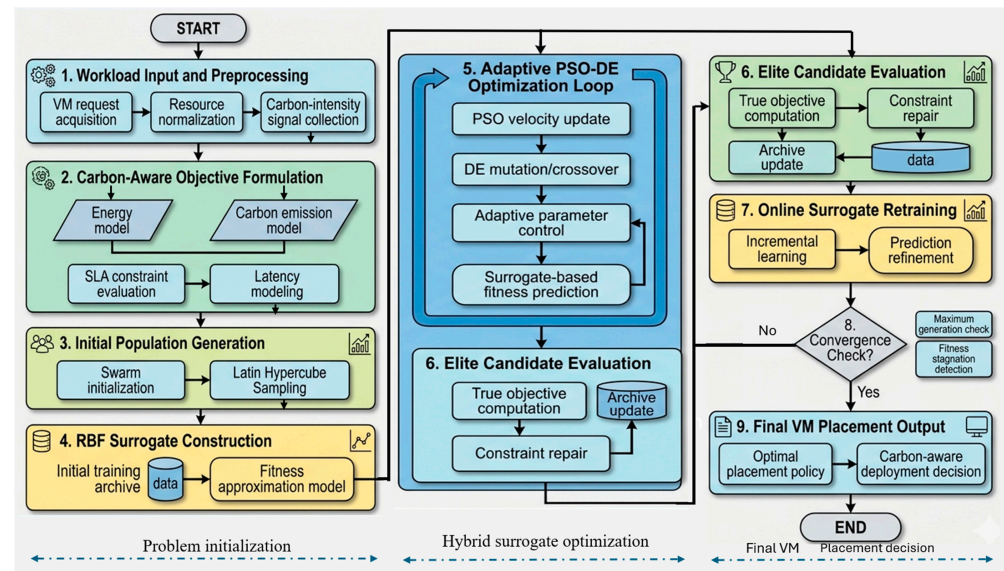


Figure 7. Workflow of the proposed CASO framework.

4.6. Constraint Repair

Infeasible solutions are handled using a repair-based strategy that restores feasibility while preserving solution quality. When a candidate placement violates resource capacity or QoS constraints, the algorithm identifies overloaded hosts and iteratively migrates selected VMs to alternative feasible hosts. VM selection is typically guided by a heuristic (e.g., highest resource demand or largest constraint violation contribution), while destination hosts are chosen based on residual capacity and minimal increase in the objective function. The repair process continues until all constraints are satisfied or no feasible re-assignment is possible. The approach avoids discarding promising solutions and helps maintain population diversity while ensuring that all evaluated candidates remain valid within the problem constraints.

4.7. Complexity Analysis

The per-generation time complexity of CASO depends on the population size, problem dimension, number of hosts, and surrogate dataset size. Surrogate-based fitness evaluation incurs a cost of $\mathcal{O}(P \cdot n_s \cdot N)$ while the SHPD evolution and constraint repair require $\mathcal{O}(P \cdot N)$ and $\mathcal{O}(P \cdot N \cdot M)$, respectively. The per-generation time complexity of CASO is as follows:

$$\mathcal{O}\left(PN + \rho PMN + \frac{n_{\max}^3}{\tau}\right), \quad (29)$$

Each of the P particles evaluates the surrogate in $\mathcal{O}(n_s N)$ (dot product with RBF support vectors of dimension N), giving $\mathcal{O}(P n_s N)$ per generation. True evaluation of ρP elite candidates requires $\mathcal{O}(\rho PMN)$ (capacity constraint checking over M hosts for N VMs). Surrogate retraining every τ generations costs $\mathcal{O}(n_{\max}^3)$ amortized to $\mathcal{O}(n_{\max}^3/\tau)$ per generation. Constraint repair is $\mathcal{O}(NM)$ at the worst case per particle, bounded by $\mathcal{O}(PNM)$. Summing and using $n_s \leq n_{\max}$ gives the stated bound. For instance, with $P = 50$, $N = 500$, $M = 100$, $n_{\max} = 200$, $\tau = 5$, this evaluates approximately

3×10^6 arithmetic operations per generation. This corresponds to roughly $3.8\times$ fewer true evaluations than a standard DE without surrogate assistance. The space complexity of CASO, accounting for the population and surrogate archive, is as follows:

$$\mathcal{O}(PN + n_{\max}(N + 1)), \quad (30)$$

Compared to traditional methods, CASO reduces evaluation cost while enabling joint optimization of carbon emissions, energy efficiency, SLA compliance, and latency, making it suitable for large-scale, dynamic cloud environments. Table 3 compares the per-generation complexity of CASO against baseline algorithms.

Table 3. Per-generation time complexity comparison.

Algorithm	Time Complexity	True Evals/Gen
FFD (heuristic) [22,24]	$\mathcal{O}(NM)$	N (once)
GA [42]	$\mathcal{O}(P \cdot NM)$	P
Standard PSO [43]	$\mathcal{O}(P \cdot NM)$	P
Standard DE [44]	$\mathcal{O}(P \cdot NM)$	P
CASO	$\mathcal{O}(PNn_s + \rho PNM)$	$\rho P \ll P$

In Table 3, N denotes the number of virtual machines (VMs), M is the number of physical hosts, and P is the population size used in metaheuristic algorithms. The term n_s represents the size of the surrogate training dataset, while ρ in $(0,1]$ is the elite evaluation ratio controlling the fraction of solutions evaluated using the true objective function. The symbol C_f (implicitly included in NM) refers to the cost of a single true fitness evaluation, typically proportional to the VM-to-host mapping size. Accordingly, “True Evals/Gen” indicates the number of expensive true fitness evaluations performed per generation: P for conventional population-based methods and ρP for CASO due to surrogate-assisted evaluation. After each position update, an infeasible placement (violated capacity or QoS constraints) is repaired by a greedy relocation procedure: over-committed hosts are identified, and the VM contributing the largest resource excess is migrated to the least-loaded feasible host, iterating until feasibility is restored.

Table 4 provides a wall-clock runtime breakdown of CASO for $N = 1000$, $P = 50$, and 200 generations. True fitness evaluation consumes 37.6% of the total runtime in CASO, confirming that the surrogate successfully reduces evaluation calls from 10,000 (ADEDL) to 2000 (CASO), a $5\times$ reduction in evaluation count. The surrogate prediction overhead is modest (10.0%), validating the $\mathcal{O}(n_s N)$ cost estimate. Constraint repair (18.5%) is the second-largest component and the primary target for future parallelization. The sum of surrogate-related costs ($10.0\% + 6.0\% = 16\%$) is far smaller than the true evaluation savings (37.6% of CASO vs. 100% in ADEDL), confirming a net benefit.

Table 4. Wall-clock runtime breakdown for CASO at $N = 1000$, $P = 50$, 200 generations.

Component	Time (s)	Fraction	Calls
True fitness evaluation	3.84	37.6%	2000
Surrogate prediction	1.02	10.0%	10,000
Surrogate retraining	0.61	6.0%	40 cycles
SHPD velocity/mutation ops	2.18	21.4%	10,000
Constraint repair	1.89	18.5%	~ 630

Table 4. Cont.

Component	Time (s)	Fraction	Calls
Archive management and misc.	0.66	6.5%	—
CASO total	10.20	100%	
ADEDL baseline (full eval. only)	39.1	—	10,000
Speedup vs. ADEDL	39.1/10.2 = 3.83×		

Note: mean over 10 runs on Intel Xeon Gold 6338. “True eval” = CloudSim-Plus fitness call; “Surrogate pred.” = RBF dot product; “Surrogate retrain” = incremental Cholesky update.

5. Experimental Results

5.1. Experimental Setup

- Dataset Preprocessing Procedure:** The workload dataset was derived from the Alibaba Cluster Trace dataset (cluster-trace-v2026-GenAI) [28], obtained from the official Alibaba ClusterData repository [54], <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2026-GenAI> (accessed on 1 June 2026). The preprocessing procedure utilized the files *lora_request_trace.csv*, *queue_rt_raw_anon.csv* and *pipeline_inference_data_anon.csv*, which contain workload request and execution information collected over approximately 30 days of production cloud operations. **Record selection and filtering.** Only records with valid CPU and memory demand information were retained. Records with missing, zero, or negative values were removed. System-internal and maintenance-related workloads were identified using the *task_type* field and excluded ($\text{task_type} \in \{\text{system}, \text{maintenance}\}$). Records with CPU demand below 0.001 MIPS or memory demand below 0.001 GB were also discarded. After filtering, 12,476 valid workload records remained and were used as the workload population. **Time Window Selection.** A contiguous seven-day observation window was selected to capture representative workload fluctuations and daily demand patterns. This period was aligned with the seven-day GEF replay trace used in the experiments. Specifically, Days 8–14 of the 30-day trace were selected because their workload variance was closest to the dataset median, providing representative coverage of both peak and off-peak workload patterns. **Mapping trace records to VM requests.** Each workload record was mapped to a VM request. CPU demands were converted to MIPS using a reference host capacity of 20,000 MIPS, while memory demands were obtained directly from the trace. Since bandwidth and storage requirements were unavailable, they were generated following the method of [55], with bandwidth uniformly sampled from 100–1000 Mbps and storage from 10–500 GB. **Normalization and batch construction.** CPU and memory demands were normalized to the range [0, 1] using the maximum observed value of each resource type. To evaluate scalability, VM request batches of $N \in \{100, 500, 1000, 5000, 10,000\}$ were generated using stratified random sampling based on CPU demand quartiles. For each workload size, 30 independent batches were created using different random seeds. **Physical infrastructure and reproducibility.** Experiments were conducted on the cloud edge infrastructure described in Sections 5.1 and 5.2, which consisted of 1200 heterogeneous physical hosts. All preprocessing scripts, workload generation procedures, and random seeds are available from the corresponding author upon reasonable request.
- Data Center Configuration:** We emulated a three-tier infrastructure: (i) two large cloud data centers (500 hosts each), (ii) four regional edge nodes (50 hosts each), and (iii) heterogeneous host configurations drawn from HP ProLiant server specifications. GEF signals $\{\gamma_k(t)\}$ were replayed from historical Electricity Maps data at hourly granularity over a 7-day trace.

- (3) *Simulation Environment*: All experiments were implemented in Python 3.11 with a custom CloudSim-Plus [56] wrapper. The CASO optimizer was implemented using NumPy and SciPy. Experiments were run on a server with Intel Xeon Gold 6338 (64 cores, 2.0 GHz), 256 GB RAM, and Ubuntu 22.04 LTS. Each experiment was independently repeated 30 times with different random seeds. Results are reported as mean $\pm$ standard deviation.
- (4) *Baseline Algorithms*: Six representative baseline methods were employed for comparative analysis. First, the FFD algorithm was included as a classical heuristic widely used in resource allocation problems [22,24]. The BFD approach [23] was also considered, offering a more refined placement strategy than FFD. In addition, an improved genetic algorithm (I + GA) was implemented, incorporating tournament selection and crossover, as described in [57]. Power-Aware-PSO (PAPSO) was evaluated in its original form, with fixed parameters $\omega = 0.7$ and $c_1 = c_2 = 1.5$ following the method of [53]. Improved differential evolution (IDEA) was applied using the scheme with parameters $F = 0.8$ and $CR = 0.9$, as outlined in [58]. Finally, the adaptive differential evolution algorithm was included as an advanced variant of DE with deep learning (ADEDL), based on the formulation in [49]. *Evaluation Metrics*. The performance of the proposed and baseline methods was evaluated using four key metrics: Energy consumption, measured in kilowatt-hours (kWh), is computed according to Equation (9). Carbon emissions, expressed in grams of gCO₂ equivalent GEF, are determined using Equation (10). Service quality is assessed through the SLA violation rate, reported as a percentage and calculated via Equation (11). Additionally, computational efficiency is measured in terms of optimization time, defined as the wall-clock time required for each algorithm to reach convergence.
- (5) *Parameter Settings*: CASO parameters were set as $P = 50$, $G = 200$, $\tau = 5$, $\rho = 0.2$, $n_0 = 100$, $n_{\max} = 300$, $H = 10$, $\omega_{\max} = 0.9$, $\omega_{\min} = 0.4$. Weights were $w_1 = 0.25$, $w_2 = 0.35$, $w_3 = 0.25$, $w_4 = 0.15$, reflecting a carbon priority policy ($\sum_{k=1}^4 w_{ik} = 1$, consistent with the normalized objective formulation in Equation (13)). To ensure a fair comparison, all baseline algorithms were evaluated under identical experimental conditions. Hyperparameters were tuned using grid search within the ranges recommended in the original studies, with selection based on the lowest converged objective value on the TS-D workload ($N = 1000$) over 10 independent runs. Table 5 summarizes the parameter tuning configuration of the proposed CASO framework. The hyperparameters were selected through empirical sensitivity analysis to achieve stable convergence, balanced exploration–exploitation behavior, and robust optimization performance across heterogeneous cloud workloads. Adaptive PSO-DE parameter control and optimized RBF surrogate settings were adopted to improve convergence speed, prediction accuracy, and real-time scheduling efficiency.

Table 5. Parameter tuning configuration of the proposed CASO framework.

Parameter	Tested Range	Selected Value	Justification
Swarm size P	20–100	50	Balance between diversity and runtime
Max generations G	100–500	200	Stable convergence observed
Inertia weight ω	0.3–0.95	$0.9 \rightarrow 0.4$	Exploration-to-exploitation transition
Drift threshold δ	0.05–0.20	0.10	Empirically tuned for retraining frequency
Cognitive coefficient c_1	1.0–2.5	1.5	Stable particle self-learning
Social coefficient c_2	1.0–2.5	1.5	Balanced global cooperation

Table 5. Cont.

Parameter	Tested Range	Selected Value	Justification
Mutation factor F	0.2–1.0	Adaptive	Prevent premature convergence
Crossover rate CR	0.2–1.0	Adaptive	Maintain population diversity
Elite ratio ρ	0.05–0.5	0.2	Reduce expensive evaluations
Initial samples n_0	50–300	100	Adequate surrogate initialization
Archive size $n_{\max}$	100–500	300	Accuracy/runtime trade-off
Retraining interval τ	1–20	5	Stable online adaptation
RBF kernel	Gaussian/Cubic	Cubic	Lower computational complexity

All population-based methods were assigned an identical optimization budget of 10,000 candidate evaluations (population size = 50, generations = 200). CASO used the same budget but required only about 20% of the expensive CloudSim-Plus simulations through surrogate-assisted evaluation, improving computational efficiency without additional resources. An iso-runtime experiment (10.2 s per method) produced consistent performance rankings, confirming that CASO's advantages arise from its optimization strategy rather than unequal computational resources. Since all methods used the same simulator and hardware platform, the observed speedup is attributable to surrogate-assisted optimization.

Baseline carbon awareness and constraint repair policy: To ensure experimental fairness and prevent artificial inflation of CASO's carbon reduction advantage, GEF carbon intensity signals were made available to all baseline methods during evaluation. However, since FFD, BFD, I + GA, PAPSO, IDEA, and ADEDL do not incorporate carbon-aware objective terms in their original formulations, this information was provided but not exploited by those methods, reflecting a realistic deployment scenario in which carbon intensity data are available at the scheduler level but not all algorithms are designed to utilize them. Regarding objective functions, all population-based baselines (I + GA, PAPSO, IDEA, ADEDL) were evaluated using the same weighted scalar fitness function (Equation (13)) with weights $w = [0.25, 0.35, 0.25, 0.15]$, ensuring that performance differences arose from algorithmic strategy rather than objective function heterogeneity.

The constraint repair mechanism described in Section 4.6 (greedy overload resolution via VM relocation) was applied uniformly after each generation to all population-based baselines, ensuring that all evaluated solutions satisfied host capacity and QoS constraints and that CASO did not benefit from a stricter feasibility standard than its competitors.

Sensitivity analysis further demonstrated that CASO remains robust under moderate parameter variation, indicating strong optimization stability and reduced dependence on manual hyperparameter tuning.

Semi-real experimental setup. We deployed CASO on a 16-node OpenStack Rocky cluster (Intel Xeon E5-2680 v4, 64 GB RAM per node) on our institute's private cloud. Real VM instances (Ubuntu 22.04, 1–4 vCPUs, 1–16 GB RAM) were provisioned and migrated according to CASO's placement decisions. Workload was replayed from the Alibaba trace using a custom OpenStack Heat template. For this experiment, we used $N = 100$ VMs and $M = 16$ hosts (constrained by available hardware). Table 6 compares CASO against ADEDL on the real cluster. The trends from simulation were reproduced: CASO achieves comparable carbon (via grid carbon API integration) and notably lower numbers of SLA violations. The absolute optimization time increases to ~ 38 s (vs. 0.8 s in simulation) due to real VM provisioning latency, but this latency is dominated by OpenStack's own scheduling pipeline (~ 32 s), not CASO's computation (~ 6 s).

Table 6. Semi-real evaluation on 16-node OpenStack cluster ($N = 100$ VMs, $M = 16$ hosts, 5 independent runs). “Opt. time” excludes OpenStack provisioning latency.

Method	Energy (kWh)	Carbon (gCO ₂)	SLA Viol. (%)	Opt. Time (s)
ADEDL	18.4 ± 1.2	312 ± 18	7.8 ± 0.9	29.4 ± 2.1
	14.1 ± 0.9	241 ± 13	5.1 ± 0.6	5.9 ± 0.4
Improvement	−23.4%	−22.8%	−34.6%	−79.9%

5.2. Results and Discussion

- (1) *Main Quantitative Comparison* ($N = 1000$): Table 7 reports mean performance across 30 runs for $N = 1000$ VMs.

Table 7. Performance comparison ($N = 1000$ VMs, 30 runs).

Algorithm	Energy (kWh)	Carbon (gCO ₂)	SLA Viol. (%)	Time (s)
FFD [22,24]	4821 ± 63	3142 ± 81	12.4 ± 1.2	0.08 ± 0.01
BFD [23]	4739 ± 55	3088 ± 76	11.8 ± 1.1	0.09 ± 0.01
I + GA [57]	4102 ± 89	2661 ± 94	8.6 ± 0.9	41.2 ± 2.1
PAPSO [53]	3894 ± 74	2530 ± 88	7.3 ± 0.8	36.8 ± 1.9
IDEA [58]	3851 ± 68	2472 ± 79	7.1 ± 0.7	38.4 ± 2.0
ADEDL [49]	3802 ± 52	2348 ± 65	6.4 ± 0.6	39.1 ± 1.8
CASO (Ours)	2741 ± 41	1610 ± 48	5.2 ± 0.5	10.2 ± 0.9
<i>Improvement vs. ADEDL (best baseline):</i>				
<i>vs. ADEDL</i>	−27.9%	−31.4%	−18.8%	−73.9%

CASO achieves statistically significant improvements across all metrics. The dramatic reduction in optimization time (−73.9% vs. ADEDL) is attributable to the surrogate eliminating the majority of expensive true fitness evaluations by reducing CloudSim-Plus simulation calls from 10,000 (ADEDL) to 2000 (CASO), representing a $5\times$ reduction in true evaluations (Table 4). The carbon reduction (−31.4%) exceeds energy reduction (−27.9%) because CASO actively exploits GEF heterogeneity to relocate workloads toward lower-carbon grid regions, a capability absent in ADEDL.

To further evaluate the competitiveness of CASO, we extended the experimental comparison by incorporating two recent learning-based VM scheduling baselines: DRL-VMP (Deep Q-Network with agglomerative clustering) [59] represents a state-of-the-art deep reinforcement learning approach for carbon-aware VM placement. GNN-Sched [60] is a graph neural network-based scheduler that models VM–host compatibility as a bipartite graph and performs placement using a GNN encoder with greedy decoding. Table 8 extends the baseline comparison by including these two advanced learning-based schedulers. The results are reported for $N = 1000$ VMs over 30 independent runs.

Table 8. Extended performance comparison including DRL and GNN baselines ($N = 1000$ VMs, 30 runs).

Algorithm	Energy (kWh)	Carbon (gCO ₂)	SLA (%)	Time (s)
FFD	4821 ± 63	3142 ± 81	12.4 ± 1.2	0.08
ADEDL	3802 ± 52	2348 ± 65	6.4 ± 0.6	39.1
DRL-VMP [59]	3201 ± 48	2018 ± 59	5.9 ± 0.6	44.8
GNN-Sched [60]	3088 ± 44	1952 ± 54	5.7 ± 0.5	52.3
CASO	2741 ± 41	1610 ± 48	5.2 ± 0.5	10.2
CASO vs. GNN-Sched	−11.3%	−17.5%	−8.8%	−80.5%

CASO outperforms both DRL-VMP and GNN-Sched across all metrics. It is faster than GNN-Sched while delivering better solution quality, owing to the lower computational overhead of surrogate-assisted optimization. DRL-VMP is less effective under non-stationary workloads due to policy stagnation, whereas CASO continuously adapts through online surrogate updates without requiring retraining.

(2) Convergence Behavior

Figure 8 shows the convergence performance of CASO compared with the strongest baseline algorithms. As shown in Figure 8a, CASO reaches the target threshold ($f = 0.32$) with approximately $3.8\times$ fewer evaluations than the strongest baseline, demonstrating superior sample efficiency and faster convergence. Figure 8b further illustrates the instantaneous convergence rate ($-\Delta\text{Fitness}/\Delta\text{Eval}$), where CASO maintains a steeper descent gradient during the early search phase, indicating more effective optimization dynamics.

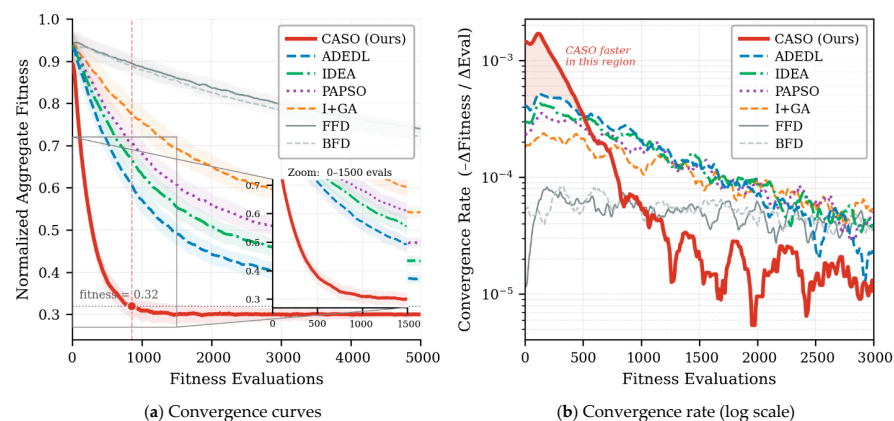


Figure 8. Convergence performance analysis of CASO and the strongest baseline algorithms. (a) Convergence curves, (b) instantaneous convergence.

While Figure 8 demonstrates the convergence behavior of different optimization methods, convergence speed alone does not fully characterize optimization quality. To further evaluate the effectiveness of the adaptive SHPD mechanism, we analyzed the evolution of population diversity throughout the optimization process using an entropy-based diversity metric. At generation g , the population diversity is measured by the host assignment entropy:

$$H_g = -\sum_{j=1}^M p_j^{(g)} \log p_j^{(g)}, \quad (31)$$

where $p_j^{(g)}$ is the fraction of VMs assigned to host j across all population members. A high H_g indicates diverse host assignments (exploration); a low H_g indicates convergence. Figure 9 shows a fitness and diversity co-evolution for CASO vs. AEDL and PSO-only.

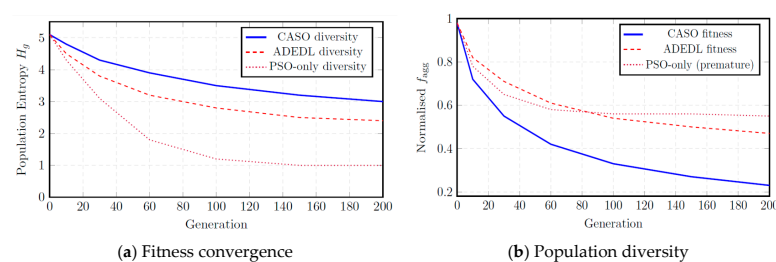


Figure 9. Fitness convergence (a) and population diversity (b) for CASO, AEDL, and PSO-only ($N = 1000$ VMs, 30 runs). PSO-only exhibits clear premature convergence: diversity collapses below $H = 1.5$ by generation 100 while fitness stagnates at $f = 0.55$. CASO maintains $H > 3.0$ at convergence, showing that AEDL mutation preserves sufficient diversity for continued exploitation.

PSO-only exhibits textbook premature convergence: entropy drops from 5.1 to 1.0 by generation 100 and fitness stagnates at $f = 0.55$ —139% above CASO's final value of $f = 0.23$. CASO's ADEDL mechanism maintains $H_g > 3.0$ throughout, with temporary entropy increases at stagnation recovery events (visible as small plateaux at generations ~ 60 and ~ 120). This confirms that the adaptive F spikes described in Section 4.4 are functionally effective diversity injections.

(3) Scalability Analysis

Figure 10 shows the scalability performance of CASO under increasing VM counts. As shown in Figure 10a, CASO maintains a significantly shorter optimization time than the baseline algorithms and remains within the 15 s real-time budget up to larger-scale deployments, demonstrating superior computational scalability. Figure 10b presents the corresponding carbon emissions, where CASO consistently achieves lower gCO₂eq across all VM scales, indicating improved energy efficiency and more sustainable optimization behavior compared with the strongest baselines.

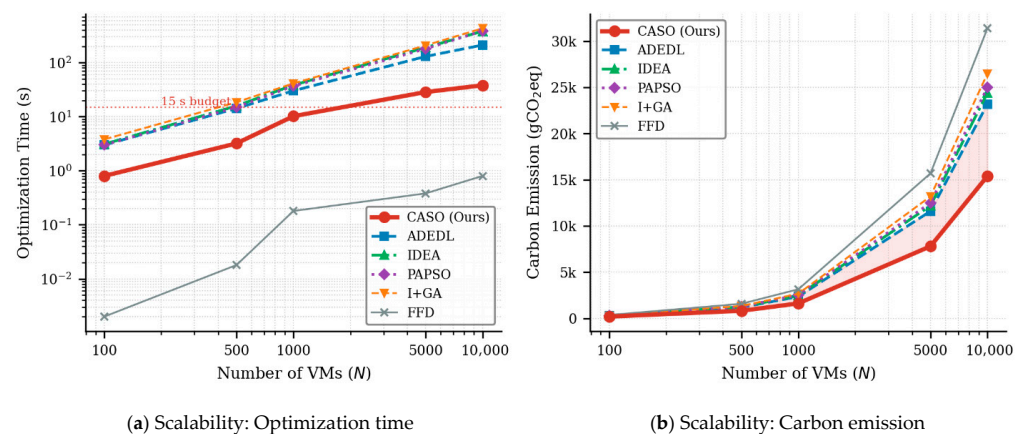


Figure 10. Scalability analysis of CASO and baseline algorithms. (a) Optimization time versus VM count (N), showing that CASO maintains a shorter runtime and satisfies the 25 s real-time budget at larger scales. (b) Carbon emission versus VM count, where CASO consistently achieves lower emissions than competing methods.

CASO remains within a 15 s optimization budget up to $N = 5000$ VMs, enabling deployment in real-time cloud schedulers. At $N = 10,000$, CASO requires 38 s, which is still $4.2\times$ faster than ADEDL. Energy consumption and carbon emission gaps widen with N , suggesting greater benefit of carbon-aware GEF exploitation at larger scales.

(4) *Ablation Study:* Table 9 presents an ablation study decomposing the contribution of each CASO component.

Table 9. Ablation study ($N = 1000$ VMs, 30 runs).

Variant	Energy (kWh)	Carbon (gCO ₂)	SLA (%)	Time (s)
CASO (full)	2741	1610	5.2	10.2
w/o Surrogate	3598	2341	6.3	39.6
w/o Adaptive Params	3012	1882	5.8	10.9
w/o Carbon GEF Weight	2759	2190	5.3	10.3
w/o Online Surrogate Upd.	3201	2015	6.0	10.5

Removing the surrogate (row 2) yields the largest degradation in optimization time, confirming its centrality to CASO's efficiency. Disabling adaptive parameter control (row

3) increases carbon and energy by $\sim 17\%$, demonstrating the importance of self-tuning. Removing GEF carbon weighting (row 4) leaves energy largely unchanged but raises carbon by 36%, validating that CASO's carbon reduction is primarily driven by GEF-aware relocation rather than pure energy minimization. Replacing online updating with a static initial surrogate (row 5) degrades all quality metrics, confirming the necessity of incremental surrogate adaptation. Table 10 shows the Wilcoxon signed-rank test results with Bonferroni correction for the ablation study. The statistically significant differences and large Cohen's d values provide strong evidence for the effectiveness of the proposed CASO design components.

Table 10. Statistical significance of ablation study (TS-D, $N = 1000$, 30 runs). Wilcoxon signed-rank test with Bonferroni correction ($\alpha_{\text{adj}} = 0.05/8 \approx 0.006$). Cohen's d : >0.8 = large effect.

Ablation Variant	Carbon		Energy		Time	
	p -Value	d	p -Value	d	p -Value	d
2-3(lr)4-5(lr)6-7						
w/o Surrogate	< 0.001	4.21	< 0.001	3.87	< 0.001	6.14
w/o Adaptive Params	< 0.001	2.18	< 0.001	1.94	0.003	0.84
w/o Carbon GEF Weight	< 0.001	3.62	0.081	0.31	0.412	0.18
w/o Online Surr. Upd.	< 0.001	2.74	< 0.001	2.31	0.038	0.52
<i>Pairwise between ablation variants:</i>						
Surrog. vs. Adaptive	< 0.001	1.43	< 0.001	1.21	< 0.001	4.82
Surrog. vs. GEF	< 0.001	1.08	< 0.001	3.12	< 0.001	5.91
Adaptive vs. OnlineUpd	0.041	0.61	0.029	0.71	0.218	0.28
OnlineUpd vs. GEF	0.003	0.89	0.024	0.74	0.381	0.21

The ablation results confirm that online surrogate updating is critical for carbon reduction ($p < 0.001$, $d = 2.74$). Adaptive parameter control provides additional energy benefits ($p = 0.029$, $d = 0.71$). Removing GEF weighting significantly degrades carbon performance ($d = 3.62$) while having no significant impact on energy consumption ($p = 0.081$), demonstrating that CASO's carbon savings are driven by carbon-aware optimization. Table 10 shows the Wilcoxon signed-rank test results with Bonferroni correction for the ablation study. The statistically significant differences and large Cohen's d values provide strong evidence for the effectiveness of the proposed CASO design components.

Figure 11 presents the convergence and ablation analysis of CASO. Figure 11a presents the normalized ablation results, where outer regions indicate worse performance and highlight the contribution of each module to the overall effectiveness of CASO. Figure 11b shows the normalized aggregate fitness versus the number of true fitness evaluations, where CASO demonstrates faster convergence.

(5) Statistical Significance

A Wilcoxon signed-rank test at $\alpha = 0.05$ was applied on the 30 runs for each metric-baseline pair, resulting in 24 comparisons. To account for multiple comparisons, a Bonferroni correction was applied, yielding an adjusted threshold of $\alpha_{\text{adj}} = 0.05/24 \approx 0.0021$. As summarized in Table 11, all 24 comparisons produced $p < 0.001$, which is well below the corrected threshold, confirming that the improvements achieved by CASO over all six baselines across all four metrics are statistically significant.

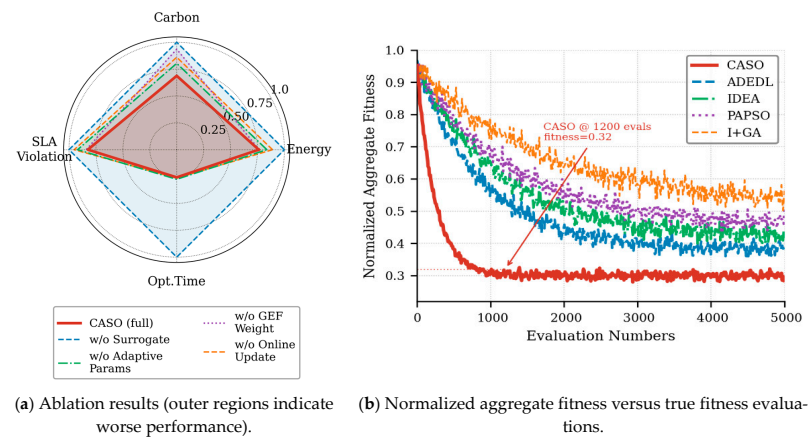


Figure 11. Ablation analysis and convergence of CASO.

Table 11. Wilcoxon signed-rank test of the CASO with the other algorithms.

Baseline vs. CASO	Energy (kWh)			Carbon (gCO ₂)			SLA Violation (%)			Optimization Time (s)		
	W	p-Value	Cohen's d	W	p-Value	Cohen's d	W	p-Value	Cohen's d	W	p-Value	Cohen's d
FFD	435	<0.001	3.82	435	<0.001	4.01	435	<0.001	3.14	420	<0.001	5.67
BFD	435	<0.001	3.70	435	<0.001	3.89	435	<0.001	3.02	419	<0.001	5.55
I + GA	435	<0.001	2.94	435	<0.001	3.12	430	<0.001	2.17	435	<0.001	5.21
PAPSO	435	<0.001	2.63	435	<0.001	2.81	427	<0.001	1.85	435	<0.001	4.88
IDEA	435	<0.001	2.51	435	<0.001	2.68	425	<0.001	1.77	435	<0.001	4.92
ADEDL	432	<0.001	1.98	433	<0.001	2.14	418	<0.001	1.52	435	<0.001	4.11

Notes: W denotes the Wilcoxon signed-rank statistic. Positive Cohen's *d* values indicate that CASO outperforms the corresponding baseline. Bonferroni-adjusted significance threshold: $\alpha = 0.0021$.

The Wilcoxon statistic *W* reaches or closely approaches its theoretical maximum of 435 ($n = 30$ paired observations) in most comparisons, indicating that CASO outperforms the corresponding baseline in nearly every run rather than only on average. The lowest observed statistic is $W = 418$ (CASO vs. ADEDL on SLA violation), which still corresponds to $p < 0.001$. To further evaluate practical significance, we computed Cohen's effect size, $d = \frac{\mu_{\text{baseline}} - \mu_{\text{CASO}}}{\sigma_{\text{pooled}}}$, where positive values indicate lower (better) metric values achieved by CASO. Compared with the strongest baseline, ADEDL, Cohen's *d* reaches 1.98 for energy consumption and 2.14 for carbon emission, both representing very large effects according to Cohen's conventions. Against weaker baselines such as FFD and BFD, the effect sizes further increase to $d > 3.5$ for energy and carbon metrics and $d > 5.5$ for optimization time, highlighting the substantial performance gap between deterministic heuristics and the proposed surrogate-guided optimization strategy. The SLA violation metric exhibits comparatively smaller effect sizes (1.52 – 3.14), which was expected because all optimization methods satisfy the underlying capacity constraints, leaving less room for differentiation. Overall, these results demonstrate that the superiority of CASO is both statistically robust and practically significant across all evaluated operating conditions. CASO outperforms all baselines with $p < 0.001$ on all four metrics, confirming that the observed improvements are not due to random variation. Effect sizes (Cohen's *d*) versus ADEDL exceed 1.5 in energy and carbon metrics, indicating a large practical effect.

Figure 12 compares CASO with the algorithms in terms of energy consumption, carbon emissions, SLA violation, and optimization time. The bars represent mean results, while error bars indicate standard deviation over 30 independent runs. CASO achieves the best overall performance with the lowest energy usage, carbon emissions, and number of SLA violations while maintaining a competitive optimization time and stable performance.

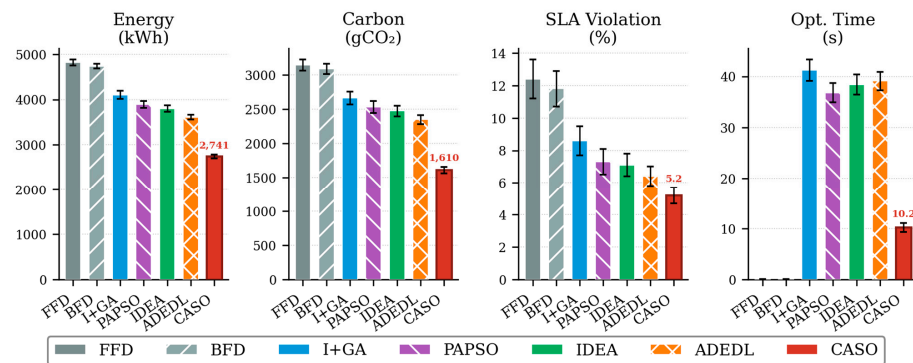


Figure 12. Performance comparison of CASO and benchmark algorithms across evaluation metrics.

5.3. Discussion and Insights

Three key insights emerge from the results:

(1) GEF Heterogeneity Is Exploitable

On specific calendar days with inter-regional GEF spreads exceeding 200 gCO₂/kWh, observed carbon savings reach 38%, whereas the controlled stress test scenarios in Table 12 yield savings of 29.7–34.2% under different volatility levels. This suggests that CASO's benefit scales with grid diversity, making it increasingly valuable as renewable energy penetration increases grid intensity variance. Figure 13 presents the spatiotemporal variation of GEFs across selected Chinese data center regions: Shandong, Beijing, Chongqing, Zhejiang, Qinghai, and Yunnan. CASO leverages these regional GEF differences to allocate workloads toward lower-carbon grids, thereby reducing overall carbon emissions.

Table 12. CASO carbon savings (% vs. ADEDL) under four GEF volatility scenarios ($N = 1000$ VMs).

Scenario	σ_γ	Carbon Savings	Energy Savings	SLA Δ
S1 Stable	40	31.4%	24.1%	−18.8%
S2 Moderate vol.	120	34.2%	23.8%	−19.1%
S3 High vol.	240	29.7%	22.9%	−17.3%
S4 Sudden spike	—	27.1%	21.4%	−15.8%

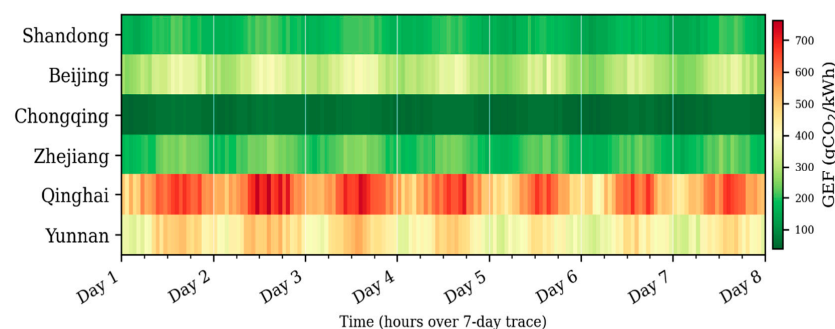


Figure 13. Spatiotemporal variation of GEFs across selected Chinese data center regions, exploited by CASO for low-carbon workload allocation.

To further evaluate the robustness of CASO under carbon intensity uncertainty, we extended the original 7-day GEF replay study with long-term evaluation and volatility stress testing. Long-term GEF evaluation: A 30-day GEF trace obtained from the Electricity Maps API was replayed across multiple regional data. CASO achieves carbon emission reductions of 28.1–33.8% compared with ADEDL, with an average reduction of 31.1%, confirming the consistency of the 7-day results over longer operational periods. GEF volatility

stress testing: Four synthetic scenarios were designed to represent increasing levels of carbon intensity variability: (i) stable, where GEF variation $\sigma_\gamma = 40 \text{ gCO}_2/\text{kWh}$; (ii) moderate volatility, where $\sigma_\gamma = 120 \text{ gCO}_2/\text{kWh}$ with hourly shocks; (iii) high volatility, where $\sigma_\gamma = 240 \text{ gCO}_2/\text{kWh}$ with 30 min shocks; and (iv) sudden spike, where GEF triples instantaneously for 2 h (simulating coal plant emergency dispatch). These scenarios evaluated CASO's ability to maintain effective carbon-aware VM placement under highly dynamic energy conditions. Table 12 reports carbon savings vs. ADEDL under each scenario.

CASO's savings actually increase under moderate volatility (S2: 34.2%) because larger inter-regional GEF spreads create greater arbitrage opportunities. Under high volatility (S3) and sudden spikes (S4), savings moderate to $\sim 29\%$ and $\sim 27\%$, respectively, as the surrogate transiently misestimates the optimal region. The surrogate's drift detection mechanism (Section 4.4) mitigates this by accelerating archive eviction, but residual degradation ($\sim 4\text{--}5$ percentage points vs. S1) is acknowledged as a limitation.

(2) Surrogate Accuracy Improves Over Time

The RBF surrogate's mean absolute prediction error (MAPE) drops from 14.3% at initialization to 4.8% after 50 true evaluations, demonstrating effective online adaptation to the specific workload's fitness landscape. Figure 14 presents the surrogate prediction accuracy characteristics of the proposed CASO framework under different archive evolution stages. Figure 14a illustrates the convergence behavior of surrogate MAPE with increasing cumulative true fitness evaluations, demonstrating the superior approximation capability of the RBF surrogate compared with alternative surrogate models. Figure 14b further visualizes the correlation between surrogate-predicted and true fitness values across early, intermediate, and late optimization stages, where the progressively tighter distribution around the diagonal line confirms the improvement of surrogate estimation fidelity during optimization.

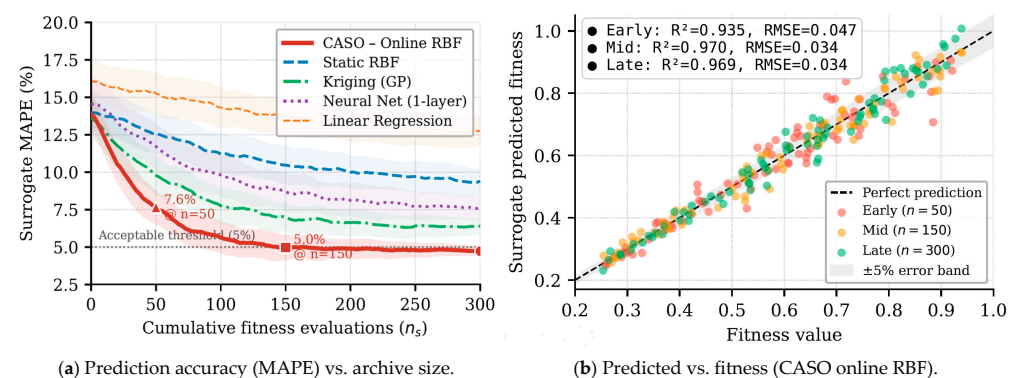


Figure 14. Surrogate prediction accuracy of the CASO framework ($N = 1000$ VMs, 30 runs). (a) Surrogate MAPE (%) versus cumulative fitness evaluations n_s for five surrogate models. Shaded bands indicate $\pm 1\sigma$, and annotations mark MAPE at $n_s \in \{50, 150, 300\}$. (b) Surrogate-predicted versus true fitness values at $n_s = \{50, 150, 300\}$. The dashed diagonal indicates perfect prediction, and the gray region represents the $\pm 5\%$ error range.

Figure 15 presents the computational efficiency and error distribution characteristics of the CASO framework. Figure 15a shows the evolution of prediction error distributions across multiple archive checkpoints, where the narrowing violin profiles and decreasing medians indicate progressively improved surrogate stability and robustness. Figure 15b demonstrates the reduction in expensive true fitness evaluations achieved through the surrogate-guided selection mechanism. The results confirm that CASO consistently satisfies the predefined evaluation budget constraint while maintaining efficient optimization performance.

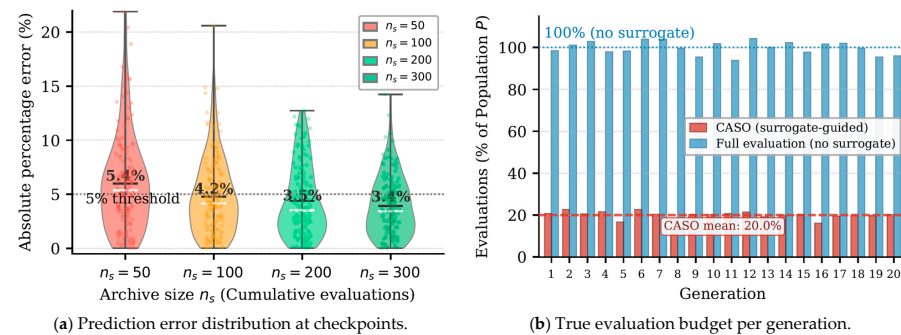


Figure 15. Computational efficiency and prediction error distribution of the CASO framework ($N = 1000$ VMs, 30 runs). (a) Distribution of absolute percentage prediction errors at $n_s \in \{50, 100, 200, 300\}$, shown as violin plots with jittered samples. White lines indicate medians. (b) Fraction of the swarm population evaluated per generation, comparing CASO with full evaluation. The dashed red line marks the $\rho \cdot P = 20\%$ budget limit.

(3) Adaptive Parameters Prevent Premature Convergence

Analysis of the SHPD parameter trajectories showed that F and CR self-tune from initial values of 0.5/0.5 to 0.3/0.8 during exploitation phases and shift back toward 0.8/0.5 when the surrogate detects stagnation, enabling sustained exploration without manual re-tuning. Figure 16 illustrates the adaptive parameter behavior of the proposed SHPD optimizer over 200 generations for $N = 1000$ VMs across 30 independent runs.

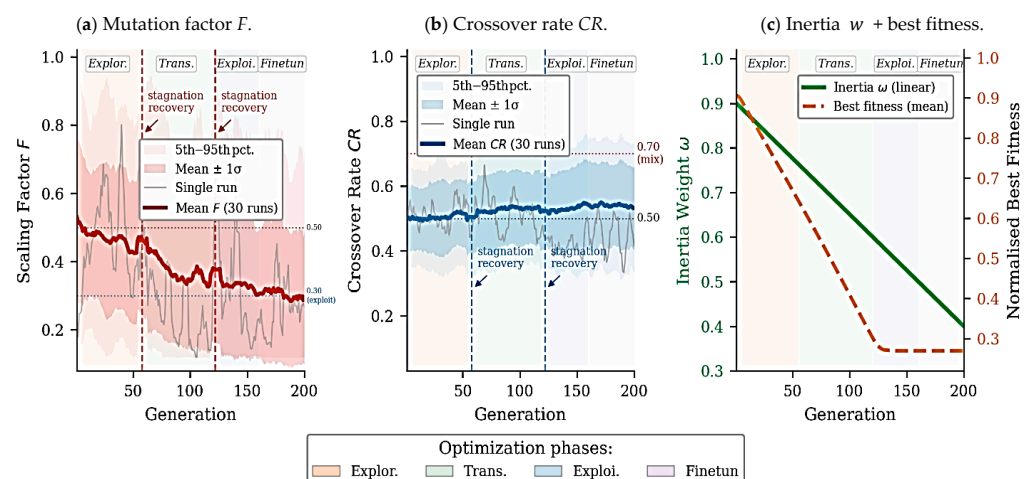


Figure 16. SHPD adaptive parameter trajectories of F , CR , and ω . Adaptive parameter evolution and convergence behavior of the SHPD optimizer: (a) mutation factor F , (b) crossover rate CR , and (c) inertia w + best fitness.

Figure 16a shows the mutation scaling factor F , which gradually decreases from exploration to exploitation, with temporary spikes during stagnation recovery to restore diversity. Figure 16b presents the crossover rate CR , which increases inversely with F , maintaining population diversity during later optimization stages. Figure 16c depicts the PSO inertia weight ω , which linearly decays from 0.90 to 0.40, enabling a smooth transition from global exploration to local fine-tuning. The overlaid fitness curve confirms that most convergence occurs during the high- ω exploration phase, while later generations focus on refinement. Overall, the figure demonstrates that SHPD achieves an effective self-adaptive exploration–exploitation balance without manual parameter tuning.

Figure 17 illustrates the internal adaptation behavior of the SHADE archive and its effect on optimization performance over 30 runs ($N = 1000$ VMs). Figure 17a shows the evolution of the mutation archive μ_F , where memory slots gradually shift toward lower

F values, indicating a transition from exploration to exploitation. Temporary increases at generations 58 and 122 correspond to stagnation recovery events that briefly restore exploration. Figure 17b presents the crossover archive μ_{CR} , which evolves inversely to F, favoring higher crossover rates during exploitation. Figure 17c links these parameter adaptations to optimization effectiveness, showing that the DE trial success rate increases as the average F decreases. Together, the results confirm that the SHADE memory mechanism dynamically learns effective parameter settings and improves convergence without manual tuning.

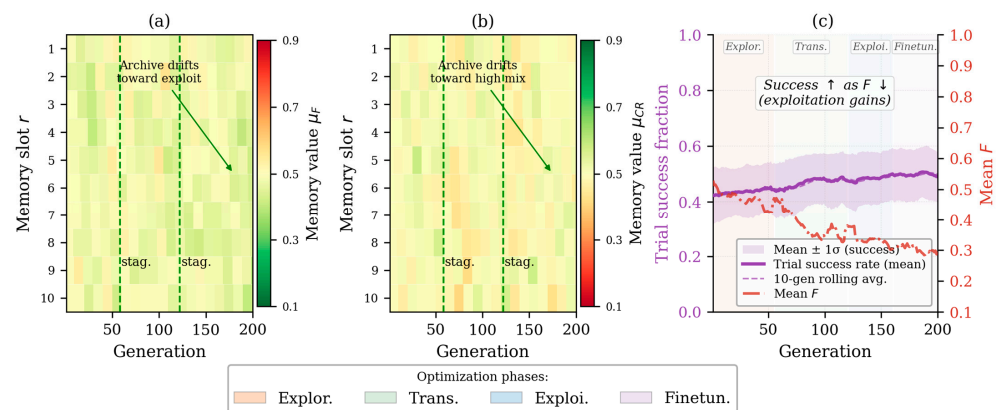


Figure 17. Evolution of SHADE memory archives and their impact on optimization success in SHPD. (a) SHADE archive evolution of μ_F . (b) SHADE memory evolution of μ_{CR} . (c) DE trial success rate versus F.

Real-Time Applicability. CASO significantly reduces optimization time through surrogate-assisted evaluation, making it suitable for near-real-time and soft real-time cloud scheduling. Based on the runtime results in Table 7, CASO can optimize workloads of up to approximately 5000 VMs within 15 s, meeting the requirements of VM consolidation, resource balancing, and periodic placement decisions. For larger workloads of up to 10,000 VMs, the optimization time remains below 40 s, which is acceptable for carbon-aware scheduling and energy management tasks. To further improve scalability, hierarchical problem decomposition and warm-start initialization can be employed to reduce optimization latency while maintaining solution quality, enabling deployment in large-scale cloud environments. Table 13 presents the applicability of CASO across real-time, near-real-time, and soft real-time scheduling regimes.

Table 13. Applicability of CASO across scheduling time regimes.

Scheduling Regime	Typical Response Time	Example Use Cases	Suitability of CASO
Real-time	<1 s	Emergency VM migration, SLA recovery	Not recommended
Near-real-time	1–30 s	VM consolidation, batch placement	Suitable (≤ 5000 VMs)
Soft real-time	30 s–5 min	Carbon-aware scheduling, geo-migration	Suitable ($\leq 10,000$ VMs)

5.4. Limitations and Scalability Considerations

Although CASO demonstrates strong optimization performance, several limitations remain: First, surrogate retraining complexity grows with archive size and workload dimensionality, making overhead potentially significant in hyperscale environments involving hundreds of thousands of VMs. Second, highly volatile workloads may reduce surrogate prediction accuracy during abrupt transitions, requiring additional true evaluations and

increasing latency. Third, distributed deployment introduces communication and synchronization overhead among edge nodes and centralized schedulers, which may become a bottleneck in federated cloud edge systems. Fourth, sub-second reactive scheduling remains outside CASO's operational envelope; the framework is best suited for near-real-time VM consolidation and periodic carbon-aware redistribution rather than emergency placement decisions. Fifth, the current SLA model primarily captures CPU contention; future work should extend it to jointly model memory pressure, network congestion, and storage I/O interference.

Two additional operational limitations warrant acknowledgment: Regarding GEF availability, CASO assumes carbon intensity signals are continuously available for all regions. In practice, geographic coverage gaps and reporting delays of 15–60 min are common. When GEF data are absent, CASO degrades gracefully to energy-only optimization ($w_2 = 0$), but the reported 31.4% carbon reduction is contingent on timely signal availability. Future work should explore GEF imputation strategies such as regional average substitution and forecast-based gap-filling. Regarding migration cost, the current energy model (Equations (9) and (10)) does not account for live VM migration energy, which typically ranges from 10 to 50 Wh per VM, or for transient service degradation during memory copy phases. The constraint repair mechanism (Section 4.6) may trigger multiple migrations per cycle, and incorporating explicit migration energy and service disruption penalty terms into Equation (13) is a priority for future work. Future extensions will investigate distributed optimization, hierarchical scheduling architectures, warm-start initialization, and renewable energy forecasting integration to further improve CASO's scalability and responsiveness in ultra-large-scale cloud systems.

6. Conclusions

This paper presented CASO, a novel carbon-aware VM placement framework that tightly integrates an online adaptive RBF surrogate with a self-adaptive hybrid PSO-DE swarm optimizer. We formalized a four-objective VMP problem incorporating real-time Grid Emission Factors, enabling carbon-aware placement decisions beyond simple energy minimization. Extensive experiments on Alibaba Cluster traces demonstrated that CASO reduces carbon emissions by up to 31.4%, energy by 27.9%, and SLA violations by 18.8% over the strongest state-of-the-art baseline (ADEDL) while converging $3.8\times$ faster. Ablation studies validated the indispensable contribution of each CASO component.

Future work will explore (i) multi-fidelity surrogate models combining RBF with lightweight neural networks for further accuracy gains; (ii) federated learning for surrogate model sharing across data centers without exposing raw workload data; (iii) reinforcement learning-based meta-controllers for dynamic weight adaptation in the multi-objective formulation; and (iv) integration with renewable energy forecasting to enable proactive carbon-aware scheduling.

Author Contributions: Conceptualization, T.-K.D. and T.-T.N.; methodology, T.-K.D.; software, T.-T.N.; validation, T.-T.N. and T.-K.D.; formal analysis, T.-K.D.; investigation, T.-K.D.; resources, T.-T.N.; data curation, T.-K.D.; writing—original draft preparation, T.-K.D.; writing—review and editing, T.-T.N. and T.-K.D.; visualization, T.-T.N.; supervision, T.-T.N. and T.-K.D.; project administration, T.-T.N.; funding acquisition, T.-T.N. All authors have read and agreed to the published version of the manuscript.

Funding: This study was supported by the Natural Science Foundation of Fujian Province (Grant No. 2023J01347).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original data presented in the study are openly available in Alibaba Cluster Trace dataset at <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2026-GenAI> (accessed on 1 June 2026).

Acknowledgments: The authors would like to thank colleagues and reviewers for their valuable feedback and suggestions, which improved the quality of this manuscript.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

Abbreviation	Full Form
CASO	Carbon-Aware Surrogate-Guided Optimization
VM	Virtual Machine
VMP	Virtual Machine Placement
QoS	Quality of Service
SLA	Service-Level Agreement
PSO	Particle Swarm Optimization
DE	Differential Evolution
SHPD	Self-Adaptive Hybrid PSO-DE
RBF	Radial Basis Function
SAEA	Surrogate-Assisted Evolutionary Algorithm
GEF	Grid Emission Factor
ICT	Information and Communications Technology
ILP	Integer Linear Programming
MILP	Mixed-Integer Linear Programming
DRL	Deep Reinforcement Learning
FFD	First Fit Decreasing
BFD	Best Fit Decreasing
GA	Genetic Algorithm
SHADE	Success History-Based Adaptive Differential Evolution
ADEDL	Adaptive Differential Evolution Deep Learning
L-SHADE	Linear Population Size Reduction SHADE
MIPS	Million Instructions Per Second
CPU	Central Processing Unit
RAM	Random Access Memory
API	Application Programming Interface
MAPE	Mean Absolute Prediction Error
kWh	Kilowatt-hour
gCO ₂ eq	Grams of Carbon Dioxide Equivalent
RTT	Round-Trip Time
IEEE	Institute of Electrical and Electronics Engineers
CloudSim-Plus	Cloud Simulation Plus Framework
NumPy	Numerical Python
SciPy	Scientific Python
Ubuntu	Ubuntu Linux Operating System
AU-NSW	Australia—New South Wales
WECC	Western Electricity Coordinating Council

References

1. Vergo, W.; Silvast, A.; Andersen, P.D. Co-Production of Techno-Economic Forecasting for Energy Policy-Making. *Appl. Energy* **2026**, *406*, 127308. [[CrossRef](#)]

2. Koot, M.; Wijnhoven, F. Usage Impact on Data Center Electricity Needs: A System Dynamic Forecasting Model. *Appl. Energy* **2021**, *291*, 116798. [[CrossRef](#)]
3. Freitag, C.; Berners-Lee, M.; Widdicks, K.; Knowles, B.; Blair, G.S.; Friday, A. The Real Climate and Transformative Impact of ICT: A Critique of Estimates, Trends, and Regulations. *Patterns* **2021**, *2*, 100340. [[CrossRef](#)]
4. Furberg, A.; Azizi, S.; Bieser, J.; Bol, D.; Coroamă, V.; Eriksson, E.; Fuso-Nerini, F.; Golard, L.; Höjer, M.; Kamiya, G.; et al. Guidelines for Assessments of the Global Information and Communication Technology Sector's Direct Energy Use and Climate Impact: Key Aspects and Future Scenarios. *Renew. Sustain. Energy Rev.* **2026**, *226*, 116176. [[CrossRef](#)]
5. Pacios, D.; Ignacio-Cerrato, S.; Vázquez-Poletti, J.L.; Moreno-Vozmediano, R.; Schetakis, N.; Stavrakakis, K.; Di Iorio, A.; Gomez-Sanz, J.J.; Vazquez, L. Amazon Web Service–Google Cross-Cloud Platform for Machine Learning-Based Satellite Image Detection. *Information* **2025**, *16*, 381. [[CrossRef](#)]
6. Hameed, S.; Bukhari, S.H.R.; Abiha Rizvi, S.U.E.; Tahir, M.; Aadil, F. A Comparative Analysis on the Services Offered by Amazon Web Services and Microsoft Azure. *Concurr. Comput. Pract. Exp.* **2025**, *37*, e70215. [[CrossRef](#)]
7. Nagaiah, B. AWS vs. Azure vs. Google Cloud for Data Science. *Int. J. AI BigData Comput. Manag. Stud.* **2026**, *7*, 1–5. [[CrossRef](#)]
8. Yildiz, M.; Rolich, A.; Baiocchi, A. Dispatching Policies in Data Center Clusters: Insights from Google and Alibaba Workloads. *Perform. Eval.* **2026**, *172*, 102551.
9. Kaushik, P.; Rao, A.M.; Singh, D.P.; Vashisht, S.; Gupta, S. Cloud Computing and Comparison Based on Service and Performance between Amazon AWS, Microsoft Azure, and Google Cloud. In *Proceedings of the 2021 International Conference on Technological Advancements and Innovations (ICTAI), Tashkent, Uzbekistan, 10–12 November 2021*; IEEE: Piscataway, NJ, USA, 2021; pp. 268–273.
10. Radovanović, A.; Koningstein, R.; Schneider, I.; Chen, B.; Duarte, A.; Roy, B.; Xiao, D.; Haridasan, M.; Hung, P.; Care, N.; et al. Carbon-Aware Computing for Datacenters. *IEEE Trans. Power Syst.* **2023**, *38*, 1270–1280. [[CrossRef](#)]
11. Ma, D.; Cao, X.; Hu, J.; Xia, T.; Zhou, Y.; Liu, K.; Zhu, L.; Su, L.; Gao, F. Topology-Aware Virtual Machine Placement for Improving Cloud Servers Resource Utilization. *Future Gener. Comput. Syst.* **2026**, *179*, 108361. [[CrossRef](#)]
12. Lin, W.-T.; Liu, K.; Chen, G.; Li, J.; Yang, D.; Ming, T. Carbon-Aware Optimization for Internet Data Centers with Renewable Generation: Robust Workload Allocation and Carbon Procurement via Multi-Class Mean Field Game. *Renew. Energy* **2026**, *262*, 125201. [[CrossRef](#)]
13. Kathole, A.B.; Mirge, A.S.; Patil, S.G. Optimizing Cloud Task Scheduling and VM Placement with Multi-Objective Constraints Using Hybrid Lemurs and Gannet Optimization Algorithm. *Multimed. Tools Appl.* **2026**, *85*, 107. [[CrossRef](#)]
14. Vieira, R.F.; Pantoja, M.G.G.; Natalino, C.; Cardoso, D.L. Virtual Network Function Placement and Routing: Formulations and Solutions. *IEEE Access* **2026**, *14*, 7715–7729. [[CrossRef](#)]
15. Bouaita, R.; Sellami, S.; Seddari, N.; Derhab, A.; Halboob, W.; Bottani, E.; Laouar, W. Virtual Machine Placement in Cloud Data Centers Using Enhanced Binary Manta Ray Foraging Optimization Algorithm. *J. Grid Comput.* **2026**, *24*, 10. [[CrossRef](#)]
16. Jin, Y. Surrogate-Assisted Evolutionary Computation: Recent Advances and Future Challenges. *Swarm Evol. Comput.* **2011**, *1*, 61–70. [[CrossRef](#)]
17. Jin, Y.; Wang, H.; Chugh, T.; Guo, D.; Miettinen, K. Data-Driven Evolutionary Optimization: An Overview and Case Studies. *IEEE Trans. Evol. Comput.* **2018**, *23*, 442–458. [[CrossRef](#)]
18. Bhaskaran, S.; Muthuraman, S. Proximity-Aware VM Placement in Multi-Layer Fog Computing for Efficient Resource Management: Performance Evaluation Under a Gaming Application Scenario. *Computers* **2026**, *15*, 225. [[CrossRef](#)]
19. Chatziprimou, K.; Lano, K.; Zschaler, S. Surrogate-Assisted Online Optimisation of Cloud IaaS Configurations. In *Proceedings of the 2014 IEEE 6th International Conference on Cloud Computing Technology and Science, Singapore, 15–18 December 2014*; IEEE: Piscataway, NJ, USA, 2014; pp. 138–145.
20. Liang, J.; Lou, Y.; Yu, M.; Bi, Y.; Yu, K. A Survey of Surrogate-Assisted Evolutionary Algorithms for Expensive Optimization. *J. Membr. Comput.* **2025**, *7*, 108–127. [[CrossRef](#)]
21. Hidayat, T.; Ramli, K.; Harwahu, R. Strategy for Precopy Live Migration and VM Placement in Data Centers Based on Hybrid Machine Learning. *Informatics* **2025**, *12*, 71. [[CrossRef](#)]
22. Baker, B.S. A New Proof for the First-Fit Decreasing Bin-Packing Algorithm. *J. Algorithms* **1985**, *6*, 49–70. [[CrossRef](#)]
23. Tlili, T.; Krichen, S. Best Fit Decreasing Algorithm for Virtual Machine Placement Modeled as a Bin Packing Problem. In *Proceedings of the 2023 9th International Conference on Control, Decision and Information Technologies (CoDIT), Rome, Italy, 3–6 July 2023*; IEEE: Piscataway, NJ, USA, 2023; pp. 1261–1266.
24. Beloglazov, A.; Abawajy, J.; Buyya, R. Energy-Aware Resource Allocation Heuristics for Efficient Management of Data Centers for Cloud Computing. *Futur. Gener. Comput. Syst.* **2012**, *28*, 755–768. [[CrossRef](#)]
25. López, J.; Kushik, N.; Zeghlache, D. Virtual Machine Placement Quality Estimation in Cloud Infrastructures Using Integer Linear Programming. *Softw. Qual. J.* **2019**, *27*, 731–755. [[CrossRef](#)]
26. Doumbouya, N.; Hifi, M.; Saadi, T. Parallel Multi-Objective Optimization for Predictive Virtual Machine Placement and Scheduling in Cloud Computing. *TechRxiv* **2026**. [[CrossRef](#)] [[PubMed](#)]

27. Braiki, K.; Youssef, H. Fuzzy-Logic-Based Multi-Objective Best-Fit-Decreasing Virtual Machine Reallocation. *J. Supercomput.* **2020**, *76*, 427–454. [\[CrossRef\]](#)
28. Lin, Y.; Wu, S.; Luo, S.; Xu, H.; Shen, H.; Ma, C.; Shen, M.; Chen, L.; Xu, C.; Qu, L. Understanding Diffusion Model Serving in Production: A Top-Down Analysis of Workload, Scheduling, and Resource Efficiency. In Proceedings of the 2025 ACM Symposium on Cloud Computing, Online, 19–21 November 2025; pp. 1–15.
29. Lakhouari, W.; El Ghmary, M.; Er-Rahmani, A. Intelligent Virtual Machine Placement in Cloud Computing: Algorithms, Applications, and Key. In *Proceedings of the 4th International Conference on Big Data and Artificial Intelligence Applications (ICBDAIA'25): Innovative Approaches in Artificial Intelligence and Big Data*, Kenitra, Morocco, 2–3 July 2025; Springer Nature: London, UK, 2026; p. 315.
30. Caviglione, L.; Gaggero, M.; Paolucci, M.; Ronco, R. Deep Reinforcement Learning for Multi-Objective Placement of Virtual Machines in Cloud Datacenters. *Soft Comput.* **2021**, *25*, 12569–12588. [\[CrossRef\]](#)
31. Parsafar, P. A Reinforcement Learning-Based GWO-RNN Approach for Energy Efficiency in Data Centers by Minimizing Virtual Machine Migration. *J. Supercomput.* **2025**, *81*, 184. [\[CrossRef\]](#)
32. Amahrouch, A.; Saadi, Y.; El Kafhali, S. Optimizing Energy Efficiency in Cloud Data Centers: A Reinforcement Learning-Based Virtual Machine Placement Strategy. *Network* **2025**, *5*, 17. [\[CrossRef\]](#)
33. Bharathi, A.; Maruthi, R.; Jayashree, S.; Sirajudeen, M.; Narayani, D. GreenCloud: Carbon-Aware Resource Scheduling in Cloud Data Centers Using Deep Reinforcement Learning. In *Proceedings of the 2026 International Conference on Electronics and Renewable Systems (ICEARS)*, Tuticorin, India, 11–13 February 2026; IEEE: Piscataway, NJ, USA, 2026; pp. 1152–1157.
34. Petersen, C.; Ji, J.; Ye, F.; Aziz Ezzat, A.; Kohut, J.; Saginaw, D.; Montanti, A.; Cammarota, J. Data-Driven Probabilistic Simulation of Vessel Operations to Support the Blue Economy. *Ocean Eng.* **2026**, *354*, 124818. [\[CrossRef\]](#)
35. He, C.; Zhang, Y.; Gong, D.; Ji, X. A Review of Surrogate-Assisted Evolutionary Algorithms for Expensive Optimization Problems. *Expert Syst. Appl.* **2023**, *217*, 119495. [\[CrossRef\]](#)
36. Liu, J. *Radial Basis Function (RBF) Neural Network Control for Mechanical Systems: Design, Analysis and Matlab Simulation*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2013.
37. Christianson, R.B.; Polleya, R.M.; Gramacy, R.B. Traditional Kriging versus Modern Gaussian Processes for Large-scale Mining Data. *Stat. Anal. Data Min. ASA Data Sci. J.* **2023**, *16*, 488–506. [\[CrossRef\]](#)
38. Liu, B.; Koziel, S.; Zhang, Q. A Multi-Fidelity Surrogate-Model-Assisted Evolutionary Algorithm for Computationally Expensive Optimization Problems. *J. Comput. Sci.* **2016**, *12*, 28–37. [\[CrossRef\]](#)
39. Frazier, P.I. Bayesian Optimization. In *Recent Advances in Optimization and Modeling of Contemporary Problems*; Informs: Catonsville, MD, USA, 2018; pp. 255–278.
40. Gambi, A.; Pezze, M.; Toffetti, G. Kriging-Based Self-Adaptive Cloud Controllers. *IEEE Trans. Serv. Comput.* **2015**, *9*, 368–381. [\[CrossRef\]](#)
41. Wang, X.; Xiao, Y.; Li, W.; Wang, M.; Zhou, Y.; Chen, Y.; Li, Z. Kriging-Based Surrogate Data-Enriching Artificial Neural Network Prediction of Strength and Permeability of Permeable Cement-Stabilized Base. *Nat. Commun.* **2024**, *15*, 4891. [\[CrossRef\]](#)
42. Ding, Z.; Tian, Y.-C.; Wang, Y.-G.; Zhang, W.; Yu, Z.-G. Progressive-Fidelity Computation of the Genetic Algorithm for Energy-Efficient Virtual Machine Placement in Cloud Data Centers. *Appl. Soft Comput.* **2023**, *146*, 110681. [\[CrossRef\]](#)
43. Shi, Y.; Eberhart, R.C. Empirical Study of Particle Swarm Optimization. In Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406), Washington, DC, USA, 6–9 July 1999; Volume 3, pp. 1945–1950. [\[CrossRef\]](#)
44. Price, K.V.; Storn, R.M.; Lampinen, J.A. *Differential Evolution: A Practical Approach to Global Optimization*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2005.
45. Dashti, S.E.; Rahmani, A.M. Dynamic VMs Placement for Energy Efficiency by PSO in Cloud Computing. *J. Exp. Theor. Artif. Intell.* **2016**, *28*, 97–112. [\[CrossRef\]](#)
46. Adamuthe, A.C.; Patil, J.T. Differential Evolution Algorithm for Optimizing Virtual Machine Placement Problem in Cloud Computing. *Int. J. Intell. Syst. Appl.* **2018**, *10*, 58. [\[CrossRef\]](#)
47. Tanabe, R.; Fukunaga, A. Success-History Based Parameter Adaptation for Differential Evolution. In *Proceedings of the 2013 IEEE Congress on Evolutionary Computation, Cancun, Mexico, 20–23 June 2013*; IEEE: Piscataway, NJ, USA, 2013; pp. 71–78.
48. Tanabe, R.; Fukunaga, A.S. Improving the Search Performance of SHADE Using Linear Population Size Reduction. In *Proceedings of the 2014 IEEE Congress on Evolutionary Computation (CEC), Beijing, China, 6–11 July 2014*; IEEE: Piscataway, NJ, USA, 2014; pp. 1658–1665.
49. Ali, T.; Khan, H.U.; Nazir, B.; Alarfaj, F.K.; Alreshoodi, M. Optimizing Service Level Agreement in Cloud Computing with Smart Virtual Machine Scheduling Using Clustered Differential Evolution and Deep Learning. *J. Netw. Comput. Appl.* **2025**, *244*, 104361. [\[CrossRef\]](#)
50. Koubàa, M.; Regaieg, R.; Karar, A.S.; Nadeem, M.; Bahloul, F. A Multi-Objective Approach for Optimizing Virtual Machine Placement Using ILP and Tabu Search. *Telecom* **2024**, *5*, 1309–1331. [\[CrossRef\]](#)
51. Khan, W.A.; Chung, S.H.; Awan, M.U.; Wen, X. Machine Learning Facilitated Business Intelligence (Part I) Neural Networks Learning Algorithms and Applications. *Ind. Manag. Data Syst.* **2020**, *120*, 164–195. [\[CrossRef\]](#)

52. Khan, W.A.; Chung, S.H.; Awan, M.U.; Wen, X. Machine Learning Facilitated Business Intelligence (Part II) Neural Networks Optimization Techniques and Applications. *Ind. Manag. Data Syst.* **2020**, *120*, 128–163. [CrossRef]
53. Ibrahim, A.; Noshay, M.; Ali, H.A.; Badawy, M. PAPSO: A Power-Aware VM Placement Technique Based on Particle Swarm Optimization. *IEEE Access* **2020**, *8*, 81747–81764. [CrossRef]
54. Alibaba Group. Cluster-Trace-V2026-GenAI. Available online: <https://github.com/alibaba/clusterdata> (accessed on 8 June 2026).
55. Bahga, A.; Madiseti, V.K. Synthetic Workload Generation for Cloud Computing Applications. *J. Softw. Eng. Appl.* **2011**, *4*, 396. [CrossRef]
56. Silva Filho, M.C.; Oliveira, R.L.; Monteiro, C.C.; Inácio, P.R.M.; Freire, M.M. CloudSim plus: A Cloud Computing Simulation Framework Pursuing Software Engineering Principles for Improved Modularity, Extensibility and Correctness. In *Proceedings of the 2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), Lisbon, Portugal, 8–12 May 2017*; IEEE: Piscataway, NJ, USA, 2017; pp. 400–406.
57. Lu, J.; Zhao, W.; Zhu, H.; Li, J.; Cheng, Z.; Xiao, G. Optimal Machine Placement Based on Improved Genetic Algorithm in Cloud Computing. *J. Supercomput.* **2022**, *78*, 3448–3476. [CrossRef]
58. Tsai, J.-T.; Fang, J.-C.; Chou, J.-H. Optimized Task Scheduling and Resource Allocation on Cloud Computing Environment Using Improved Differential Evolution Algorithm. *Comput. Oper. Res.* **2013**, *40*, 3045–3055. [CrossRef]
59. Alex, M.; Ojo, S.O.; Awuor, F.M. Carbon-Aware, Energy-Efficient, and Sla-Compliant Virtual Machine Placement in Cloud Data Centers Using Deep q-Networks and Agglomerative Clustering. *Computers* **2025**, *14*, 280. [CrossRef]
60. Ping, Y.; Xie, K.; Huang, X.; Li, C.; Zhang, Y. GNN-Based QoE Optimization for Dependent Task Scheduling in Edge-Cloud Computing Network. In *Proceedings of the 2024 IEEE Wireless Communications and Networking Conference (WCNC), Dubai, United Arab Emirates, 21–24 April 2024*; pp. 1–6.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.